Ministère de l'Enseignement supérieur et de la Recherche scientifique

Université de la Manouba

Institut Supérieur des Arts Multimédia de la Manouba ISAMM

Projet de Fin d'étude

Développement d'une approche de maintenance prédictive basée sur l'IA et l'IOT pour améliorer les performances des moteurs électriques dédiés aux équipements industriels.

Réalisé par

Mejri Mehdi - MIME

Encadré par

Mr. Brik Kais - ISAMM

Mr. Grati Khaled - SUP'COM

Année universitaire 2024-2025

Remerciement

Mes plus vifs remerciements vont à toutes les personnes qui ont contribué au bon déroulement et à l'aboutissement de ce rapport de stage de fin d'études, réalisé dans le cadre de ma licence en micro- informatique et machines embarquées.

Je remercie sincèrement Monsieur Kais Brik, professeur à l'Institut Supérieur des Arts et Multimédia (ISAMM), mon encadrant académique, pour sa confiance, son accompagnement rigoureux et ses précieux conseils tout au long de cette expérience.

Je tiens également à exprimer ma profonde gratitude à Monsieur Khaled Grati, maître de stage à Sup'Com, pour son accueil chaleureux, son suivi quotidien, sa disponibilité et son encouragement constant durant ce projet de fin d'études.

Je remercie chaleureusement l'ensemble de mes enseignants de l'ISAMM pour la qualité de leur enseignement, qui m'a permis d'acquérir les compétences nécessaires à la réalisation de ce stage.

Enfin, j'adresse mes remerciements à tout le personnel de Sup'Com, avec qui j'ai eu le plaisir de collaborer et de tisser d'excellentes relations professionnelles, pour leur aide précieuse et leur soutien durant cette période.

Sommaire

Introduction générale 8

Chapitre 1 : Présentation du labo de recherche « GRES'COM » et cadre du projet 10

1....Présentation du labo GRESCOM 11

1.1. Introduction 11

1.2. GRES'COM 11

1.2.1. Historique du laboratoire 11

1.2.2. Programme de recherche..... 12

2....Description du projet.....13

2.1. Contexte général 13

2.2. Problématique.....15

2.3. Objectifs de projet..... 16

3....Conclusion..... 18

Chapitre 2 : Développement d'une approche de maintenance prédictive.....19

1....Introduction 20

2....Principe de maintenance prédictive 21

3....Développement de l'approche 21

4....Maintenance des moteurs électriques..... 23

5....Etude Conceptuelle.....26

5.1. Flux de données et chaîne de valeur.....26

5.2. Algorithmes prédictifs et critères de performance..... 27

5.3. La Méthode efficace pour traiter un signal (Signal Sampling)..... 27

6....Partie logicielle..... 29

6.1. Environnement de développement..... 29

6.2. Acquisition et traitement 29
6.3. Construction et intégration du modèle 30
6.4. Communication sans fil 31
7....Partie matérielle 31
7.1. Plateforme STEVAL-STWINBX1 31
7.2. Architecture capteurs 32
7.3. Schéma de câblage 32
7.4. Configuration de la collecte temps réel 33
8....Spécification des besoins 33
8.1. Besoins Fonctionnels 33
8.2. Besoins Non Fonctionnels 35
8.3. Besoins Matériels 36
9....Conclusion 37

Chapitre 3: Réalisation et Evaluation 38
1....Introduction 39
2....Environnement de travail 39
2.1. Couche physique 39
2.2. Couche Connexion 41
2.3. Couche application 43
3....Mise en œuvre expérimentale 45
3.1. Montage du banc de test 45
3.2. Déploiement du Firmware 47
3.3. Automatisation du build et des tests 48
4....Acquisition des données et génération du modèle TinyML 49
4.1. Collecte des données avec HSDatalogger GUI 49
4.2. Montage physique du STWIN.box sur le moteur et génération des données d'anomalie 50
4.2.1. Installation Mécanique Du STWIN.box 50

4.2.2. Simulation de Données..... 52
4.3. Prétraitement des données.....55
4.4. NanoEdge AI Studio : Création du projet et benchmark automatique..... 57
4.5. Génération et intégration de la bibliothèque C..... 59
5....Projets STM32CubeIDE : Acquisition temps réel et inférence TinyML..... 60
5.1. Configuration du fichier.ioc.....60
5.2. Code d'acquisition et d'inférence (exemple le capteur d'accélération ISM330DHCX) 61
5.3. Projet STM32CubeIDE : NetXDuo MQTT & BLE pour la supervision..... 63
5.3.1. Configuration NetXDuo pour MQTT.....63
5.3.2. Configuration BLE GATT pour notifications.....65
6....Tests fonctionnels..... 66
6.1. Scénarios de test.....66
6.2. Mesures de performance 67
7....Analyse des résultats.....68
7.1. Comparaison aux exigences.....68
7.2. Points forts et axes d'amélioration 68
7.3. Perspectives d'évolution..... 69
8....Evaluation..... 69
8.1. Comparaison avec les systèmes de maintenance prédictive étudiés.....69
8.2. Tests des besoins fonctionnels et non fonctionnels 70
9....Conclusion..... 70

Conclusion..... **71**

Bibliographie.....**72**

Liste des Tableaux

Tableau II.1 – Étude et critique de l'existant 25
Tableau II.2 – Paramètres d'intervalle glissant et taux de recouvrement par capteur 28
Tableau II.3 – Paramètres d'échantillonnage et tailles de fenêtre par capteur 28
Tableau II.4 – Tailles de fenêtre et recouvrements par capteur 29
Tableau III.1 – Matériels utilisés (couche physique) 41
Tableau III.2 – Matériels utilisés (couche connexion) 43
Tableau III.3 – Matériels utilisés (couche application) 44

Liste des Figures

Figure I.1 : Schéma des axes de recherche du GRESCOM.....13
Figure I.2 : Diagramme de la vue générale des projets du laboratoire GRESCOM et met en évidence la position de mon projet de fin d'études.....13
Figure II.1 – Vue Générale sur le Dattalogger2..... 30
Figure II.2 – Les composant de kit de développement STWIN.box..... 32
Figure III.1 – Différents outils logiciels utilisés pour le développement, accompagnés de leurs logos respectifs.....40
Figure III.2 – Différents outils logiciels utilisés pour le développement, accompagnés de leurs logos respectifs.....42
Figure III.3 – Différents outils logiciels utilisés, accompagnés de leurs logos respectifs..... 44
Figure III.4 – Assemblage du STWIN.box : insertion de la batterie LiPo et fermeture du boîtier par cinq vis.....46
Figure III.5 – STWIN.box et STLINK-V3MINIE : connecteur de programmation compatible STLINK-V3. 47
Figure III.6– Flash du Firmware du STWIN.box via STLINK-V3MINIE.....48
Figure III.7 – Les étapes nécessaires des configurer le « Serial Monitor » PUTTY 49
Figure III.8 – L'acquisition des données de 11 capteurs différents en temps réel et configuration des paramètres nécessaire (cas normal et anormal)..... 50
Figure III.9 – STWIN.box et le moteur a courant continue avant le Montage physique 51
Figure III.10 – STWIN.box et le moteur a courant continue après le Montage physique..... 52
Figure III.11 – Simulation d'anomalies sur axes X, Y, Z et acquisition temps réel (ISM330DHCX) 54
Figure III.12 – Simulation de freinage par obstacle et acquisition en temps réel (accéléromètre ISM330DHCX).....54
Figure III.13 – Exemple de signal du capteur « ism330dhcx_gyroscope » à traiter.....55
Figure III.14 – Comparaison signal normal vs signal anormal (IIS3DWB Vibromètre).....56
Figure III.15 – Comparaison signal normal vs signal anormal (gyroscope ISM330DHCX).....56
Figure III.16 – Dans mon cas il faut choisir « detect anomalies »..... 57

Figure III.17 – Traitement à l'aide du IA.....58
Figure III.18 – Télécharger une Benchmark..... 58
Figure III.19 – après la validation il y a l'entrainement du modèle IA en temps réel.....59
Figure III.20 – les fichier après le déploiement, (NanoEdgeAI.h et libneai.a)..... 60
Figure III.21 – Output de Data en temps réel du capteur de vibration IIS3DXB..... 61
Figure III.22 – Extrait C ISM330DHCX-ACC utilisant le FIFO pour accélérer l'apprentissage IA..... 62
Figure III.23 – IA en temps réel pour détection locale d'anomalies.....62
Figure III.24 – Serveur MQTT opérationnel et publication des alertes sur topics dédiés..... 64
Figure III.25 – Connexion Wi-Fi et envoi sécurisé d'alertes d'anomalie (STM32CubeIDE)..... 65
Figure III.26 – Étapes de jumelage – STWINBX1 et ST BLE Sensor (Bluetooth) – Acquisition sans fil en temps réel via Bluetooth..... 66

Introduction générale

Dans le cadre de ma licence en Micro-Informatique et Machines Embarquées (IoT) à l'ISAMM, j'ai réalisé mon Projet de Fin d'Études (PFE) au sein de l'École Supérieure des Communications de Tunis (Sup'Com). Ce stage, effectué entre février et mai 2025 (durée : 4 mois), a porté sur la mise en place d'une solution embarquée de maintenance prédictive et de détection d'anomalies appliquée à un banc d'essai moteur. Le projet a consisté à :

J'ai intégré un environnement matériel basé sur le kit STEVAL-STWINBX1, équipé d'un microcontrôleur STM32U585AI et de capteurs de vibration, courant et acoustique pour la collecte de données en temps réel. À l'aide de NanoEdge AI Studio, j'ai développé et entraîné un modèle d'analyse d'anomalies validant la solvabilité fonctionnelle du moteur, puis je l'ai déployé sur STM32CubeIDE. Lorsque le modèle détecte une anomalie en local, les alertes sont transmises en temps réel via les modules Wi-Fi MXCHIP EMW3080 et Bluetooth Low Energy vers un serveur local ou une application mobile. Enfin, j'ai validé l'ensemble sur le banc d'essai en confrontant les prédictions aux scénarios de défauts (désalignement, déséquilibre, usure de roulements, blocage mécanique, etc.). Le lien avec mon projet était double : d'une part, renforcer mes compétences en traitement de signal et en apprentissage automatique embarqué ; d'autre part, comprendre les processus d'intégration matérielle et logicielle propres à l'embarqué industriel.

Pour contourner les problèmes liés à maintenance prédictive des moteurs électriques, Le laboratoire GRES'COM nous propose d'aborder la thématique intitulée : « Développement d'une approche de maintenance prédictive basée sur l'IA et l'IOT pour améliorer les performances des moteurs électriques dédiés aux équipements industriels »

Le rapport est subdivisé en 3 chapitres :

Le premier chapitre situe le cadre général de notre étude en présentant en premier lieu l'organisme d'accueil l'École Supérieure des Communications de Tunis (SUP'COM), sa réputation d'excellence et le pôle Maintenance Prédictive et Intelligence Embarquée où s'est déroulé mon PFE. En second lieu présentation de la problématique et la solution proposé illustrant ainsi le contexte institutionnel et technique de la solution développée sur STWIN.box avec NanoEdge™ AI Studio.

Dans le deuxième chapitre nous élaborons une démarche qui vise à Présenter en premier lieu une démarche qui détaille la méthodologie retenue, les choix d'outils et l'architecture de la solution acquisition multi-capteurs (vibration, courant, acoustique, etc.), entraînement et déploiement du modèle embarqué, puis transmission en temps réel des alertes via BLE et Wi-Fi pour une supervision globale.

Dans le troisième chapitre, nous analysons en première étape l'environnement technique et organisationnel, depuis le montage du banc d'essai jusqu'à l'automatisation des procédures de compilation et de test. Ensuite, il expose la génération du modèle TinyML, la configuration matérielle du STWIN.box et l'analyse des résultats expérimentaux validant l'efficacité de la maintenance prédictive embarquée.

CHAPITRE 1 :

Présentation du labo de recherche « GRES'COM » et cadre du projet

1. Présentation du labo GRESCOM

1.1. Introduction

Le Projet de Fin d'Études (PFE) vise à familiariser l'étudiant avec le monde professionnel et à renforcer ses compétences techniques et organisationnelles. Dans ce cadre, j'ai réalisé mon PFE à l'École Supérieure des Communications de Tunis (SUP'COM), du 01 février au 30 Mai 2025. Couronnée par une solide réputation et une longue tradition d'excellence, SUP'COM offre un environnement d'apprentissage dynamique et stimulant pour les étudiants désireux de se plonger dans le monde passionnant des technologies de communication. Fondée en 1998 et membre de la Conférence des Grandes Écoles, SUP'COM dispose de laboratoires GRES'COM et de structures de recherche modernes, propices aux projets innovants en systèmes embarqués et IoT.

Ce PFE s'est déroulé au sein du pôle Maintenance Prédictive et Intelligence Embarquée, où j'ai conçu et déployé une solution de détection d'anomalies sur un banc d'essai moteur à l'aide du kit STWIN.box (STEVAL-STWINBX1), une plateforme de prototypage dédiée aux applications de maintenance prédictive embarquées. Cette immersion m'a offert l'opportunité de mettre en œuvre NanoEdge™ AI Studio pour l'entraînement de modèles d'apprentissage automatique embarqué, et d'appliquer mes connaissances en traitement du signal et communication sans fil.

1.2. GRES'COM: (Green & Smart Communication Systems)

1.2.1. Historique du laboratoire

Le laboratoire GRESCOM a été créé en 2011 en associant des chercheurs spécialistes dans diverses thématiques des technologies de l'information et de la communication. Ils se sont associés pour :

- Mettre au point de nouvelles topologies de systèmes radio logicielle et définir des techniques de traitement cognitives permettant de transformer le système de communication physique en une plate-forme informatique programmable et intelligente à base d'objets distribués intelligents :
- Proposer de nouvelles architectures matérielles à très haute intégration et reconfigurables et de nouvelles techniques de programmation embarquée permettant l'implémentation de

solutions complexes et coopératives de systèmes embarqués pour les nouvelles applications nécessitant une multiplicité de traitement temps réel et à haute performance ;

- Concevoir des dispositifs optoélectroniques et circuits photoniques intégrés pour le développement de nouvelles approches de transmission tout optique, du traitement quantique de l'information et du stockage à haute densité d'information.

1.2.2. Programme de recherche

Le programme de recherche de GRESCOM vise la promotion de l'excellence de la recherche appliquée et l'innovation en TIC (Technologies de l'Information et de la Communication) opérée en Tunisie, contribuer au rayonnement international de l'université tunisienne et accroitre la participation de nos chercheurs dans des programmes et projets de recherche académiques et industriels.

Le programme est défini de façon à associer les compétences scientifiques de nos chercheurs et l'expertise technologique et métier des partenaires du monde socio-économique et des centres de recherche internationaux, sur des thèmes répondant aux enjeux mondiaux dans le domaine des TIC et leurs applications dans les secteurs jugés Prioritaires pour la Tunisie. Ainsi, et dans le but de faciliter la valorisation des résultats obtenus les actions de recherche et d'innovation du laboratoire seront inscrites dans une démarche de satisfaction des besoins économiques ou sociétaux à moyen et long terme.

L'ambition scientifique est, à partir de la mise en commun des compétences des chercheurs de GRESCOM et de nos partenaires académiques et industriels de renom international, de continuer la conduite de projets de recherche permettant d'allier la recherche d'idées originales aux aspects de valorisation pour l'usage du numérique dans diverses applications de développement d'infrastructures réseau de communication hétérogène, robuste et à haute efficacité énergétique ainsi que la mise au point d'équipements d'extrémité (Edge Devices) intégrant des techniques d'intelligence artificielles pour la perception autonome et le contrôle intelligent en tant que facilitateurs des services novateurs de l'Internet du futur dont notamment des objets. Un intérêt particulier est également accordé à la cyber sécurité basée sur les technologies classiques et quantiques pour proposer des schémas innovant répondant aux exigences de sécurité des systèmes de communication.

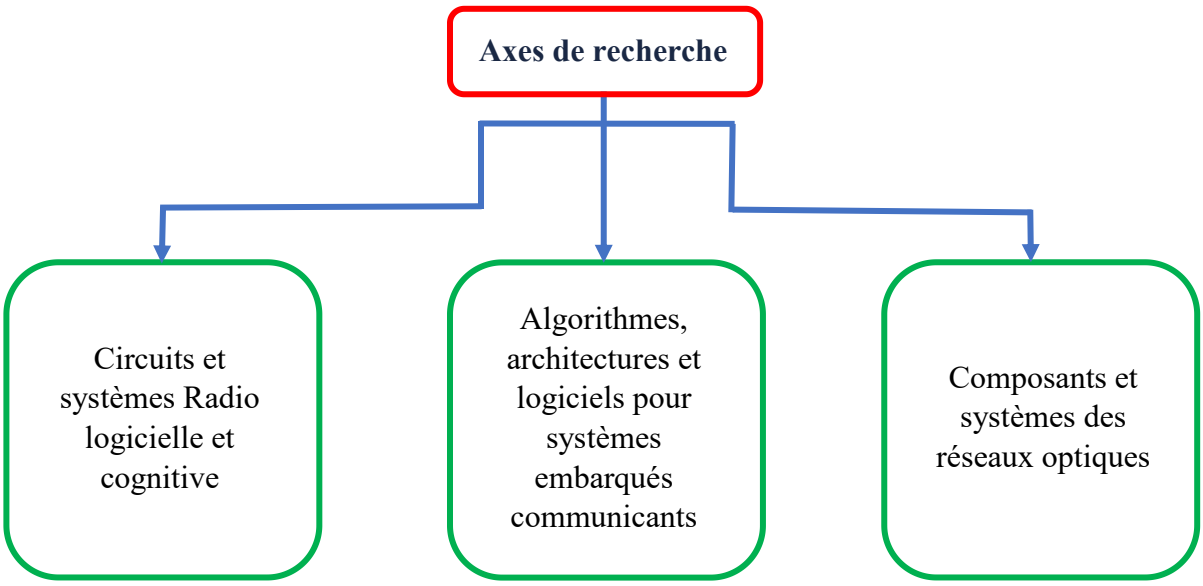

Figure I.1 : Schéma des axes de recherche du GRESCOM

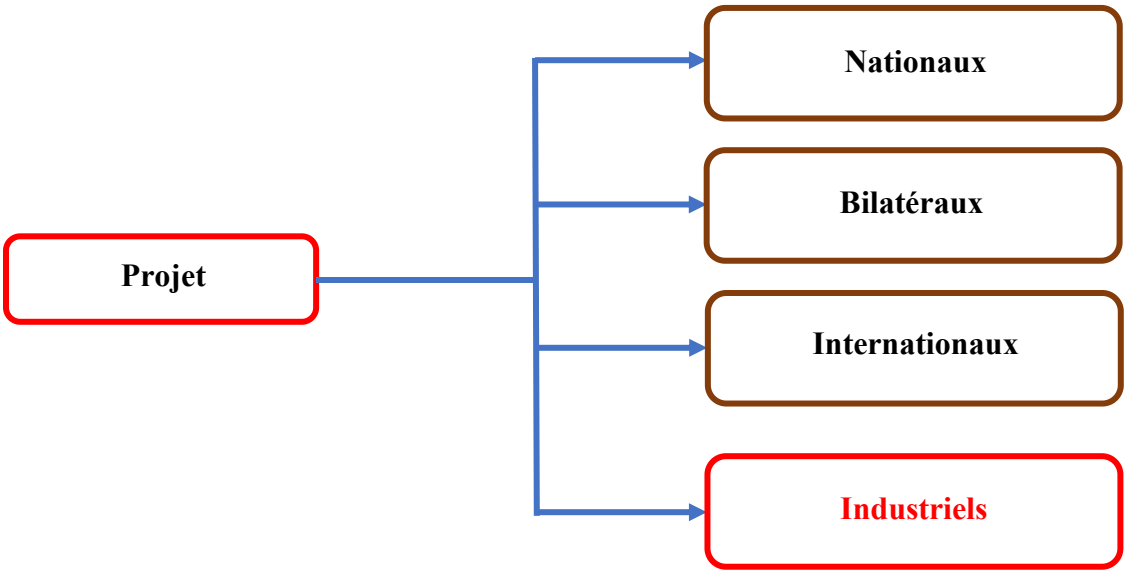

Figure I.2 : Diagramme de la vue générale des projets du laboratoire GRESCOM et met en évidence la position de mon projet de fin d'études

2. Description du projet

2.1. Contexte général

Dans un univers où la fiabilité et la performance des équipements sont primordiales, la maintenance prédictive se présente comme une des solutions permettant d'anticiper les

pannes et d'optimiser la durée de vie des machines, bien différente de la maintenance corrective suivant le modèle qui interviendrait a posteriori de la survenance d'une défaillance, ou de la maintenance préventive suivant le modèle reposant sur des intervalles de temps figés, et qui fait appel à des technologies avancées pour surveiller en temps réel l'état de santé des équipements et détecter leurs anomalies avant qu'elles engendrent des pannes. Ce modèle repose sur l'analyse de données, fournies par des capteurs embarqués permettant de déterminer des variations de fonctionnement anormales dans un moteur comme des variations excessives

Ou des écarts trop importants de température ou des bruits d'un fonctionnement anormal, par exemple, et dans l'utilisation d'intelligence artificielle et d'EDGE Computing qui viendraient Renforcer ce diagnostic avec des modèles d'auto-apprentissages adaptés à la surveillance de modèles de systèmes bien particuliers.

La **maintenance prédictive** trouve ses applications dans plusieurs secteurs :

- Dans **le domaine de l'industrie**, elle est devenue un outil essentiel à la **Smart Factory** et à **l'Industrie 4.0**. Des grandes entreprises de fabrication, de production d'électricité, de transports emploient des systèmes de surveillance intelligente pour prévenir des arrêts de production coûteux et pour améliorer la sécurité des travailleurs. Par exemple, dans une usine, **la détection précoce d'un défaut** sur **un moteur électrique** permet d'anticiper **une panne** qui entrerait de nombreux arrêts de la chaine de production.
- Dans le secteur **des bâtiments** et **des maisons intelligentes**, **la maintenance prédictive** commence à jouer un rôle de plus en plus important pour un fonctionnement optimum des équipements du quotidien. Les climatiseurs, les ascenseurs, les systèmes de chauffage mais aussi les appareils électroménagers peuvent **être surveillés en temps réel**, afin de **prévenir les pannes**, mais aussi mieux gérer la consommation énergétique.
- Dans le secteur des transports, pour les voitures électriques comme pour les trains ou les avions, la maintenance prédictive est **un outil clé en main pour optimiser les capacités d'une machine** tout en réduisant le risque de pannes en pleine activité. Les constructeurs automobiles ont d'ores et déjà adopté cette technologie dans le but de rendre leur véhicule encore plus sûr.

➢ Vers un Avenir connecté et intelligent

Avec le développement de l'IOT (internet des objets), de l'Intelligence artificielle et des microcontrôleurs embarques, la maintenance prédictive est de plus en plus accessible et performante. Elle permet de diminuer à la fois les couts de maintenance et d'accroître la durabilité des équipements et de diminuer leur impact environnemental.

Mon projet s'inscrit dans cette logique et d'une part cherchera à développer une solution embarquée sur STM32U585AI qui, d'une part, analysera en temps réel les signaux de capteurs installés sur un moteur pour détecter les anomalies grâce à NanoEdge AI Studio et d'autre part

Transmettra les alertes détectées à un serveur local par le biais d'une connexion Wi-Fi ou Bluetooth. Une perspective qui fait bien écho au monde plus intelligent et autonome que l'on envisage à l'avenir pour l'industrie et pour le monde domestique.

2.2. Problématique

Avec la vitesse de l'évolution technologique et l'extension des compétences automatisées, moteurs et machines ont pris dans beaucoup de domaines – industrie, transport, électroménager, etc. – une place essentielle, que ce soit à l'échelon d'un moteur de production dans une usine, d'un moteur dans une automobile ou d'un simple appareil ménager comme un mixeur ou un aspirateur. Tous ces périphériques au service de l'homme se trouvent exposés à une usure continue ; usure qui, si elle n'est pas détectée au bon moment, peut se transformer en panne imprévisibles, causant des pertes de temps et de ressources financières et parfois, occasionnant les dangers.

Deux stratégies majeures d'entretien équipaient les entreprises :

- **La Maintenance corrective**, consistant à réaliser des interventions sur défaillance constatée ; bien que parfois considérée comme moins lourde financièrement dans le court terme, elle engendre souvent des surcoûts liés à l'urgence des réparations fortuites et des immobilisations de machines.
- **La Maintenance préventive**, prenant en compte des intervalles de contrôle et entretien fixes, permettant de moins subir d'imprévus mais laissée encore à la prise de « températures » relatives, et parfois de synoptique réactionnel.

Devant ces diverses limites, il devient de fait nécessaire d'adopter une démarche plus intelligente : **La Maintenance Prédictive**. En tirant parti de l'intelligence artificielle, des algorithmes de traitement et de l'analyse des données en temps réel, il est dorénavant permis

de suivre l'état de fonctionnement des moteurs et des machines afin d'appréhender les premiers signes d'un éventuel défaut quelque temps avant que ne survienne la panne.

Mon projet entre pleinement de ce point de vue en posant la question centrale de :

- **Comment construire une solution embarquée capable d'analyser en permanence l'état d'un moteur ou d'une machine, d'anticiper d'éventuels dysfonctionnements et ainsi d'optimiser la gestion de la maintenance pour éviter les arrêts imprévus et prolonger la durée de fonctionnement des matériels ?**

L'enjeu étant de taille puisqu'il s'agit d'un problème lié aussi bien aux grandes industries qu'aux systèmes embarqués dans les véhicules, en passant par les applications domestiques. Prévoir une rupture nous aiderait non seulement à rendre les machines plus fiables, mais également à mieux contrôler leurs coûts d'entretien, et à les **faire fonctionner davantage sans panne**. Mon projet consiste ainsi à mettre au point une solution à base **d'un microcontrôleur STM32U585AI** incluant des capteurs intelligents et un modèle d'**IA**, que nous construirons avec **NanoEdge AI Studio**, afin d'effectuer du suivi en temps réel, d'identifier les moindres **variations anormales du comportement d'un moteur**, et **d'émettre des alertes via Wifi ou Bluetooth vers un serveur local**.

Par une approche plus astucieuse et plus réactive, ma réflexion s'inscrit déjà dans la promesse de **l'industrie 4.0**, des véhicules connectés ou de la maison intelligente.

D'un point de vue technique et économique, loin d'être négligeable, la maintenance prédictive paraît nécessaire car au-delà d'aider à la mise en œuvre d'une gestion optimisée et économe de la maintenance elle permet à toutes les entreprises de diminuer leurs couts d'exploitation, éviter les pertes de productions dues à des arrêts imprévus et favoriser la rentabilité des équipements. Elle prolonge **la durée de vie des moteurs** et évite de gaspiller les ressources dans une logique de développement durable, réduisant ainsi l'empreinte carbone des équipements dont on limite le remplacement des composants usés. Autant de préoccupations qui ne regardaient hier que les militants écologistes sont devenus aujourd'hui fondamentales pour toutes les industries concurrentielles en quête d'innovation pour optimiser leur ressource.

2.3. Objectifs de projet

C'est la raison pour laquelle le projet a pour but d'aboutir à la conception d'une solution complète et intelligente pour avoir la possibilité d'assurer la surveillance de l'état de moteurs

Et machines (le moteur est ici pris en tant qu'exemple), la montée de l'automatisation et des objets connectés entraîne la nécessité de surveiller et d'anticiper les défaillances des moteurs des moteurs et machines, objectif qui devient de plus en plus important en matière industrielle (machines d'usines), transports (motrices des trains, véhicules), appareils domestiques (ventilateurs, lave-linge). Mon projet s'inscrit dans cette démarche et propose de travailler sur la conception d'une solution embarquée capable d'analyser en temps réel l'état d'un moteur, puis de déceler les anomalies, et d'alerter les utilisateurs avant que la panne ne survienne.

Pour juger de la faisabilité de cette solution plusieurs objectifs sont définis :

- Concevoir et développer un système embarqué communiquant à l'aide du **microcontrôleur STM32U585AI,** présent sur la Carte **STEVAL-STWINBX1**, permettant de collecter en continu les données des capteurs en vue de surveiller les états de fonctionnement d'un moteur ou d'un équipement.
- Intégrer un ensemble de capteurs de qualité supérieure, **accéléromètres**, **magnétomètres**, **température** et **microphones** afin d'analyser les vibrations, le bruit et la chaleur du moteur, afin de détecter **un comportement anormal** et prévenir d'un dysfonctionnement.
- Modéliser la solution d'intelligence artificielle embarquée **NanoEdge AI Studio** capable de détecter les écarts par rapport à un fonctionnement normal et d'améliorer avec le temps sa capacité d'apprentissage en mode incrémental.
- Pour assurer l'efficience de communication des données, on dispose de la connectivité **Wi-Fi** et **Bluetooth** du module **MXCHIP EMW3080** qui va transmettre **les alertes** et rapports d'analyse à un serveur local, de telle sorte que les opérateurs puissent suivre le bon fonctionnement des appareils sans accès à un cloud ou à la connexion internet.
- Pour développer une interface de surveillance intuitive, elle doit permettre, à l'opérateur ou à l'usager final, d'accéder en **temps réel** au diagnostic, d'analyser l'évolution de l'état des moteurs et de recevoir des alertes au moment **d'un risque de panne**.
- Il faut optimiser **la consommation d'énergie** du système, en limitant la consommation, étant donné le niveau de contrainte, en assurant l'énergie de fonctionnement des dispositifs sur le **long terme**, même sous des conditions d'environnement contraint.

- Réaliser le test et la validation de **la solution** sur divers **moteurs** et **machines**, appareils électroménagers comme mixer aspirateur, moteurs industriels, machines embarquées sur les véhicules.

  Le projet a pour ambition de proposer une réponse fiable et innovante, et qui s'adapte à toutes configurations possibles, et qui participe à la transition vers un entretien plus efficace, plus préventif et plus optimal, grâce à l'intégration de l'embarqué et du traitement intelligent des données, cette solution pourrait **permettre de prolonger la durée de vie des équipements, réduire les coûts de maintenance et d'améliorer la sécurité des utilisateurs**.

3. Conclusion

La maintenance prédictive apparaît comme une méthode indispensable pour prévoir les pannes et améliorer la disponibilité des appareils. Avec son expertise en **systèmes embarqués et IoT**, le laboratoire **GRESCOM** a créé un cadre propice à la réalisation d'un prototype innovant basé sur la carte **STWIN.box** et le microcontrôleur **STM32U585AI**. La solution mise au point, combinant l'acquisition multisensorielle, le traitement sur site et l'apprentissage **TinyML** via **NanoEdge™ AI Studio**, a la capacité de détecter en temps réel les premiers signes d'anomalies et de générer des notifications sans fil, tout en tenant compte des contraintes liées à la latence, aux ressources disponibles et à la consommation énergétique. Ce travail illustre concrètement la valeur ajoutée de **l'Edge Computing** et de l'intelligence artificielle pour transformer les enjeux industriels actuels en opportunités d'efficacité et de durabilité.

CHAPITRE 2 :

Développement d'une approche de maintenance prédictive

1. Introduction

Dans le monde des usines, le temps où les robots bossent, comparé aux moments où ils sont censés bosser, c'est super important. Ça montre à quel point tout roule et si les machines tiennent le coup sans faire de caprices bizarres.

Un haut degré d'accès se change en plus de succès, une façon plus futée d'utiliser les choses et une vraie baisse des dépenses de création.

La disponibilité influence non seulement la productivité, mais également la qualité des produits finis et la satisfaction client : chaque minute d'arrêt non planifié se répercute sur la chaîne logistique, la planification des ressources humaines et l'image de marque de l'entreprise. Face à des marchés toujours plus exigeants, maintenir un taux de disponibilité proche de 100 % est devenu un objectif stratégique pour limiter les goulots d'étranglement et maximiser le retour sur investissement des installations.

Les arrêts imprévus créent des frais bruts plutôt forts : d'après une analyse d'Aberdeen Research, le prix banal d'un arrêt d'une ligne de production peut monter jusqu'à 260 000 $ par heure.

En plus de ces frais, il y a des dépenses cachées, comme une amende bizarre si on ne livre pas à temps, une image de marque abîmée, des clients fâchés et des occasions ratées.

Dans des coins bizarres, le choc des sous peut taper jusqu'à 11 % des ventes annuelles d'une boîte du top 500, un pactole de presque 1,5 billions de dollars à l'année.

La maintenance prédictive, ou « condition-based maintenance », se distingue de la maintenance préventive et planifiée en s'appuyant sur l'analyse en continu de paramètres clés (vibrations, température, acoustique, etc.) pour estimer la dégradation d'un équipement et prévoir le moment optimal d'intervention, seulement lorsque cela est nécessaire et avant la défaillance. Cette approche permet de transformer des arrêts imprévus, coûteux et

Désorganisateurs, en arrêts planifiés plus courts et mieux préparés, tout en prolongeant la durée de vie des actifs et en optimisant la gestion des pièces de rechange.

- Acquérir en temps réel des données multi-capteurs (vibration, acoustique, température, Vitesse, etc.).
- Intégrer et exécuter un modèle d'intelligence artificielle sur NanoEdge™ AI Studio pour détecter et prédire les anomalies de fonctionnement.
- Générer des alertes locales et transmettre via BLE ou Wi-Fi un état « normal » ou « anomalie » à un serveur local pour une supervision globale.

Ce cadre posé, le chapitre 2 détaillera dans un premier temps la méthodologie retenue, avant de présenter les choix d'outils et, enfin, l'architecture complète de la solution.

2. Principe de la maintenance prédictive

La maintenance prédictive s'appuie sur l'examen constant et immédiat des données provenant de capteurs intégrés (par exemple, vibrations, température, acoustique, etc.) pour prévoir les défaillances avant leur manifestation. Au lieu de faire face à des interruptions inattendues ou de planifier des réparations sur la base d'horaires prédéfinis, ce modèle astucieux utilise des algorithmes de traitement du signal et d'apprentissage automatique pour identifier les premiers indices de détérioration ou de défaillance. Ainsi, la maintenance ne sert plus uniquement à répondre aux exigences concrètes de l'équipement, mais contribue aussi à augmenter la disponibilité des machines, réduire les coûts opérationnels et atténuer l'impact écologique, en intervenant seulement lorsque le danger d'une panne dépasse un seuil critique.

3. Développement de l'approche

❖ **Description de l'approche générale de maintenance prédictive**

Ma solution proposée s'appuie sur le module **STWIN.box (STEVAL-STWINBX1),** considéré comme le cœur de notre solution. Il s'agit d'une plateforme à très basse consommation, intégrant le **microcontrôleur STM32U585AI**, et un grand nombre de capteurs industriels, parfaitement adaptée à la maintenance prédictive et à la surveillance des conditions.

➢ **Les capteurs embarqués et leur rôle**

- **IIS3DWB** (capteur de vibrations Large Bande) : permettant de capter les vibrations jusqu'à 6 kHz, est extrêmement utile pour déceler les déséquilibres ou défauts dans les roulements.
- **ISM330DHCX** (IMU 6 DoF avec ML Core) : est un accéléromètre et gyroscope 3 axes, qu'il est possible de faire fonctionner dans le cas de chocs ou de variances de mouvements non conformes.
- **IIS2DLPC** (accéléromètre 3 axes hautes performances) : assurer des mesures de vibrations exactes en permanence.
- **IIS2MDC** (magnétomètre 3 axes) : en complément permet de détecter d'éventuelles interférences magnétiques ou des décalages de positions.

- **IIS2ICLX** (inclinomètre 2 axes avec ML Core) : cellule permettant d'identifier des variations d'inclinaison, qui peuvent être un indicateur de désalignement mécanique.
- **ILPS22QS** (barographe) : surveille la pression absolue, qui dans certains environnements industriels, permet d'identifier des fuites ou obstructions.
- **STTS22H** (capteur de température) : pour mesurer la température locale avec une précision de 0.5°C, permet d'identifier précocement un échauffement.
- **IMP34DT05** (microphone MEMS PDM) et **IMP23ABSU** (microphone analogique) : analysent les signatures acoustiques du moteur pour détecter les bruits anormaux (cognements, grincements)

➢ **Acquisition de données en temps réel**

L'architecture de la STWIN.box me permet de collecter constamment toutes ces mesures à haute fréquence sans avoir besoin d'un stockage local important :

- Les capteurs transmettent leurs données directement au **STM32U585AI**,
- Un **Datalogger FP-SNS-DATALOG2** peut être utilisé pour des essais terrain, mais mon système reste orienté **streaming** vers le moteur IA,
- Cette acquisition rapide et synchrone est cruciale pour fournir au modèle IA des trames complètes, permettant une détection d'anomalie fine et réactive.

➢ **Rôle majeur de NanoEdge™ AI**

J'ai formé un modèle d'anomalie sur des séquences de fonctionnement « normales » en utilisant NanoEdge AI Studio. Cela donne lieu à une bibliothèque C intégrée qui :

- **Apprend in situ** (on-Devices training) pour s'adapter aux spécificités de chaque moteur,
- **Infère en temps réel** les écarts par rapport au comportement appris.
- Fonctionne avec **quelques dizaines d'exemples seulement**, sans expertise data science. Ainsi, l'IA devient capable de signaler un « anormal » dès l'apparition de la première variation critique.

➢ **Notifications via MQTT (Wi-Fi)**

▪ **Pilote Wi-Fi et pile TCP/IP**

- Le module EMW3080 est contrôlé par des **AT-Commands** pour se connecter au réseau local.
- La gestion du **TCP/IP** et du **protocole MQTT** est assurée par **Azure RTOS NetX Duo**, une pile réseau embarquée optimisée pour les applications IoT et temps réel.

- **Connexion à Azure IoT Hub**

- Grâce à NetX Duo, mon STM32U585AI agit comme un **client MQTT** v3.1.1.
- Il établit une connexion sécurisée (TLS ou non, selon configuration) vers **Azure IoT Hub**, qui fait office de **broker MQTT**.
- Les messages d'anomalie sont publiés sur des **topics prédéfinis** avec **QoS 1**, garantissant leur livraison et évitant toute perte d'alerte.
- Le module **MXCHIP EMW3080** est piloté en AT-Commands pour se connecter au Wi-Fi,
- Il publie les messages d'alerte sur un **broker MQTT local**,
- Les clients (application MQTT web/mobile) sont abonnés aux topics d'anomalie et reçoivent immédiatement les notifications

➢ **Diffusion et supervision via Bluetooth (BLE)**

Parallèlement, j'utilise la connectivité **Bluetooth Low Energy** intégrée :

- Le **Firmware** implémente le **BlueST Protocol**,
- L'application **ST BLE Sensor** (Android/iOS) affiche en temps réel les données capteurs et les statuts d'anomalie,
- Cette double diffusion (MQTT + BLE) garantit une **surveillance locale et distante** sans latence excessive

4. Maintenance des moteurs électriques

Les machines intelligentes sont partout, rendant leur fiabilité essentielle. Les méthodes classiques de maintenance, souvent rigides et centralisées, ne tiennent pas compte de l'état réel des équipements. L'IA embarquée et l'IoT, via des microcontrôleurs STM32 et NanoEdge AI Studio, permettent désormais une détection locale des anomalies en temps réel, sans passer par le cloud. Dans ce contexte, j'ai utilisé la carte STWIN.box et ses capteurs (vibrations, son, etc.) pour identifier les défaillances d'un moteur et envoyer des alertes immédiates via Wi-Fi ou Bluetooth.

Ainsi, ce projet se positionne entre deux univers :

- D'un côté, les plateformes industrielles puissantes mais coûteuses.
- De l'autre, une **solution embarquée, légère, autonome**, parfaitement adaptée à l'**industrie 4.0**, à la **domotique**, et aux **systèmes embarqués intelligents**.

Cette étude montre les limites des méthodes classiques et m'a conduit à concevoir un système intelligent et adaptable pour anticiper les anomalies.

- **General Electric – Moteurs d'avion :**

GE équipe ses turbines de capteurs (température, vibrations, pression) pour analyser en continu les données et détecter les anomalies avant toute panne, optimisant ainsi le calendrier de maintenance.

  - J'applique le même principe à une échelle réduite, en analysant les signaux d'un moteur pour anticiper ses défaillances.

- **Siemens – Chaînes de production intelligentes :**

Siemens équipe ses usines de capteurs (vibrations, température) pour détecter automatiquement les anomalies et déclencher des alertes.

  - Cela prouve que ma solution peut optimiser la maintenance aussi bien sure de grandes lignes de production que sur des équipements compacts (par ex. une cafetière Expresso).

- **Bosch – Appareils électroménagers connectés:**

Bosch propose des appareils électroménagers (lave-linge, etc.) dotés de capteurs détectant des anomalies (pompe bouchée, bruits anormaux) et alertant l'utilisateur via une application mobile, pour intervenir avant panne.

  - Cela montre qu'on peut intégrer de l'intelligence dans des objets du quotidien, comme mon système le fait pour un mixeur ou un aspirateur.

- **Tesla – Véhicules intelligents:**

Tesla équipe ses véhicules de nombreux capteurs qui surveillent en continu l'état de la voiture. Dès qu'un composant montre un début de défaillance, le système l'identifie, alerte le conducteur et peut même corriger certains problèmes par mise à jour logicielle.

  - J'applique la même logique, à plus petite échelle : surveillance en temps réel, détection précoce des anomalies et facilitation des actions correctives.

- **Schneider Electric – Gestion des bâtiments:**

Schneider Electric équipe les bâtiments tertiaires de capteurs pour surveiller ascenseurs, ventilations et tableaux électriques, et anticiper les pannes avant qu'elles ne deviennent critiques.

  - Cette démonstration montre que ma solution prédictive, modulaire et connectée, s'adapte aussi bien à l'industrie qu'aux bâtiments.

Nom de l'entreprise	Secteur d'application	Technologie de connexion	Composants IoT utilisés	Disponibilité en Tunisie	Lien avec mon projet	Spécifications / Détails
General Electric	Aéronautique – moteurs d'avion	Réseaux industriels / Satellite	Capteurs de température, pression, vibration, système embarqué	Non (technologie propriétaire)	Mon système, bien qu'à plus petite échelle, repose sur le même principe de collecte et d'analyse de signaux moteurs pour anticiper les défaillances.	GE surveille en temps réel les moteurs d'avion via des capteurs avancés. Les données sont analysées pour planifier les maintenances avant toute panne critique.
SIEMENS	Industrie – chaînes de production	Réseaux industriels / Ethernet	Capteurs vibratoires, thermiques, automates industriels	Partielle (via Siemens Tunisie)	Ma solution montre qu'une technologie embarquée, même légère, peut optimiser la maintenance prédictive sur des lignes de production à petite ou moyenne échelle.	Siemens utilise des capteurs pour détecter l'usure des machines, déclencher des alertes et améliorer le cycle de vie des équipements industriels.
BOSCH	Électroménager intelligent	Wi-Fi / Bluetooth	Capteurs acoustiques, moteurs, appli mobile intégrée	Oui (produits disponibles localement)	Le principe d'intégrer des capteurs dans des appareils du quotidien m'inspire directement pour adapter mon système à un mixeur ou un aspirateur.	Les appareils Bosch détectent les anomalies mécaniques (pompe bouchée, bruit anormal) et envoient des alertes à l'utilisateur avant toute panne.
TESLA	Automobile – véhicules connectés	LTE / Wi-Fi / Bluetooth	Capteurs embarqués, ECU, télémétrie, IA	Non direct (modèles importés)	Même si je travaille sur des équipements plus simples, le principe reste le même : analyse en temps réel, détection d'anomalies, et réaction anticipée.	Tesla intègre une maintenance prédictive via analyse continue de données véhicule, détection d'erreurs et même correction à distance par mises à jour logicielles.
Schneider Electric	Bâtiments – gestion des équipements	Réseaux locaux / IoT / Wi-Fi	Capteurs de courant, pression, température, interface cloud	Oui (via partenaires locaux)	Mon architecture embarquée pourrait être utilisée dans ce type d'environnement pour surveiller de petits moteurs ou circuits critiques dans les bâtiments.	Schneider surveille ascenseurs, moteurs de ventilation, tableaux électriques, et déclenche des interventions avant panne grâce à une analyse intelligente.

Tableau II.1 : Différentes entreprises de Maintenance

Le contexte actuel montre une grande diversité d'approches à visée prédictive utilisées par des grands groupes industriels dans des secteurs variés. Le tableau II.1 présente une étude comparative détaillé de ces différentes entreprises

5. Etude Conceptuelle

Cette partie offre une idée folle de la vérification d'état, montrant comment elle diffère des façons de faire habituelles et préventives, explique tout le parcours des données, de la mesure au signal d'alarme, et dévoile les règles pour choisir les formules de prédiction ainsi que les chiffres importants pour bien l'utiliser sur STM32U585AI via NanoEdge AI Studio.

5.1. Flux de données et chaîne de valeur

1. **Collecte de données via capteurs**

Des capteurs industriels (accéléromètres, microphones, capteurs de courant, thermocouples) sont placés aux points critiques de la machine pour acquérir en continu les signaux bruts.

2. **Prétraitement sur Cortex-M33**

Le microcontrôleur STM32U585AI (Cortex-M33) réalise un premier traitement embarqué (filtrage, normalisation, transformée de Fourier) afin de réduire le bruit et de préparer les données pour l'inférence.

3. **Benchmark et génération de la bibliothèque NanoEdge AI**

NanoEdge AI Studio prend en entrée les exemples de signaux prétraités et les paramètres MCU (mémoire, fréquence) pour sélectionner automatiquement la combinaison optimale de méthode de prétraitement, d'algorithme ML (kNN, SVM, réseaux de neurones, ou algorithmes propriétaires) et d'hyperparamètres.

4. **Inférence TinyML embarquée**

La bibliothèque statique générée est intégrée dans le code embarqué via STM32CubeIDE, permettant une inférence en temps réel (détection, classification ou régression) directement sur le microcontrôleur sans besoin de cloud.

5. **Transmission et supervision locale**

En cas de détection d'anomalie, le STM32U585AI émet une alerte via BLE ou Wi-Fi (MXCHIP EMW3080) vers un serveur local pour archivage, visualisation et déclenchement d'actions correctives planifiées.

5.2. Algorithmes prédictifs et critères de performance

Sélection des méthodes

- **Détection d'anomalie uninominale (one-class) :** adaptée quand seules des données « normales » sont disponibles.
- **Classification supervisée :** employée si l'on dispose de jeux de données étiquetés « normal » vs « anomalie ».
- **Évaluation de la durée de vie résiduelle :** Dans le cadre de mon projet, il s'agit d'estimer le temps restant avant une défaillance à partir de données temporelles issues des capteurs en temps réel.

NanoEdge AI Studio examine attentivement une gamme étendue d'options signal-algo-hyperparamètres pour identifier la bibliothèque idéale.

Critères de performance

- **Taux de vrais positifs (TPR) et taux de faux positifs (FPR) :** C'est pour trouver le juste équilibre entre la capacité à repérer efficacement les anomalies et la maîtrise des fausses alertes – un facteur essentiel pour garantir la fiabilité opérationnelle de la solution de la détection.
- **Durée d'inférence :** évaluer le temps nécessaire pour traiter chaque échantillon ; ce critère est déterminant en contexte embarqué temps réel, où chaque milliseconde peut faire la différence.
- **Empreinte mémoire :** volume de la bibliothèque et des buffers, restreint sur Cortex-M33 aux ressources limitées.
- **Utilisation d'énergie :** en lien direct avec la latence et l'utilisation du processeur, influençant l'autonomie des appareils embarqués.

L'équilibre entre ces indicateurs – souvent contradictoires – conditionne le succès d'une solution TinyML de maintenance prédictive sur microcontrôleur.

5.3. Traitement d'un signal dans la Maintenance

La procédure du traitement d'un signal pour effectuer une maintenance fiable est basée sur 3 étapes :

- **Étape 1 : Identifier les fréquences d'échantillonnage (Hz)**

Chaque capteur possède une fréquence d'échantillonnage spécifique (f_s), qui correspond au nombre de fois par seconde où une mesure est enregistrée. Cette information est essentielle pour

adapter le traitement des données en fonction du type de capteur utilisé. Le tableau ci-dessous résume les fréquences typiques des capteurs présents sur la carte STWIN.box.

Capteur	Nom complet	Fréquence d'échantillonnage typique (f_s)
Microphone	IMP34DT05	16 kHz (16 000 Hz)
Accéléromètre + Gyro	ISM330DHCX	416 Hz
Inclinomètre	IIS2ICLX	200 Hz
Magnétomètre	IIS2MDC	100 Hz
Capteur de température	STTS22H	1 Hz
Baromètre (pression)	ILPS22QS	1 Hz

Tableau II.2 – Paramètres d'intervalle glissant et taux de recouvrement par capteur

- **Étape 2 : Choix de la taille de fenêtre (-SL)**

La taille de fenêtre (-sl) correspond au nombre d'échantillons traité à chaque analyse. Elle dépend de:

- La **fréquence d'échantillonnage** du capteur (f_s)
- La **durée désirée** de chaque segment d'analyse

Formule :

$$-SL = \text{Durée de fenêtre (s)} \times FS$$

En pratique, on choisit souvent une fenêtre comprise entre **1 et 2,5 secondes** (ajustable selon l'application).

- **Valeurs recommandées de -sl**

Capteur	Fréquence d'échantillonnage (f_s)	Fenêtre 1 s (-sl = 1 × f_s)	Fenêtre 2 s (-sl = 2 × f_s)
Microphone (IMP34DT05)	16 000 Hz	-sl 16000	-sl 32000
Accéléromètre + Gyro (ISM330DHCX)	416 Hz	-sl 416	-sl 832
Inclinomètre (IIS2ICLX)	200 Hz	-sl 200	-sl 400
Magnétomètre (IIS2MDC)	100 Hz	-sl 100	-sl 200
Température (STTS22H)	1 Hz	-sl 1	-sl 2
Baromètre (ILPS22QS)	1 Hz	-sl 1	-sl 2

Tableau II.3 - Paramètres d'échantillonnage et tailles de fenêtre par capteur

- **Étape 3 : Choix de l'intervalle glissant (–si)**

L'intervalle glissant (–si) détermine le degré de recouvrement entre deux fenêtres successives.

Formule :

−\Si=Facteur de recouvrement−\SL

- **Valeurs recommandées pour –si (recouvrement à 50 %)**

Capteur	Fenêtre 1 s (–sl)	Recouvrement 50 % (–si = –sl/2)	Recouvrement 80 % (–si = –sl/5)
Microphone	16 000	8 000	3 200
Accéléromètre + Gyro	416	208	83
Inclinomètre	200	100	40
Magnétomètre	100	50	20
Température	10	5	2
Baromètre	10	5	2

Tableau II.4 – Tailles de fenêtre et recouvrements par capteur

6. Partie logicielle

6.1. Environnement de développement

- **STM32CubeIDE :** IDE basé sur Eclipse®/CDT™ et la chaîne d'outils GCC, offrant configuration des périphériques, génération de code, compilation et débogage GDB pour Firmware en C/C++ sur STM32.
- **NanoEdge™ AI Studio :** outil STMicroelectronics guidant l'utilisateur à travers la collecte de données, l'annotation, le benchmark automatique et la génération de bibliothèques TinyML pour détection d'anomalies, détection, classification et régression.
- **Scripts Python :** ils automatisent la collecte (« datalogging ») des signaux normaux et défectueux et offrent des outils de visualisation (Pandas, Matplotlib), ce qui simplifie l'exploration des séries temporelles et la préparation des jeux de données.

6.2. Acquisition et traitement

- **Interface SPI2 (Example ISM330DHCX) :** configuration des interruptions (ISR) et du DMA en mode circulaire pour capturer les trames brutes à 6 667 Hz, garantissant un échantillonnage en continu sans surcharger le processeur grâce à l'utilisation d'un buffer circulaire.

- **ADC1 (IMP23ABSU) :** acquisition analogique sur PC0/PC1, mise en place d'un DMA à double buffer pour numériser le signal acoustique avec un rapport signal/bruit élevé (64 dB, 130 dBSPL max).
- **PDM (IMP34DT05) :** conversion PDM→PCM intégrée via la bibliothèque STM32 PDM2PCM, alimentée par un DMA en mode circular pour échantillonnage temps réel jusqu'à 16 kHz.
- **Bus I2C2 et I2C3 :** ces interfaces maîtres sont configurées pour interroger en continu des capteurs additionnels (température, courant, etc.), avec prise en charge des interruptions et des délais d'attente (timeouts) afin de préserver la stabilité du bus en cas de blocage.
- **Buffer circulaire :** un mécanisme clé déployé pour chaque capteur, qui assure une collecte ininterrompue et synchronisée des données, sans aucune perte — une condition indispensable pour des inférences fiables en temps réel.
- **USART2 / USART3 (Debug & Console) :** canaux série dédiés à l'affichage de journaux (logs), à la navigation dans des menus interactifs et à la calibration dynamique du système, facilitant le suivi et le réglage "à chaud".

6.3. Construction et intégration du modèle

• **Datalogger Python :** Dans le cadre de mon projet, ce script autonome collecte en continu les séries temporelles « normal » et « anormal » pour chaque capteur, les étiquette, puis les sauvegarde au format CSV (NanoEdge accepte ce type de fichier .CSV). En parallèle, il génère automatiquement des graphiques de tendance et de spectre, ce qui facilite l'exploration et l'interprétation des données.

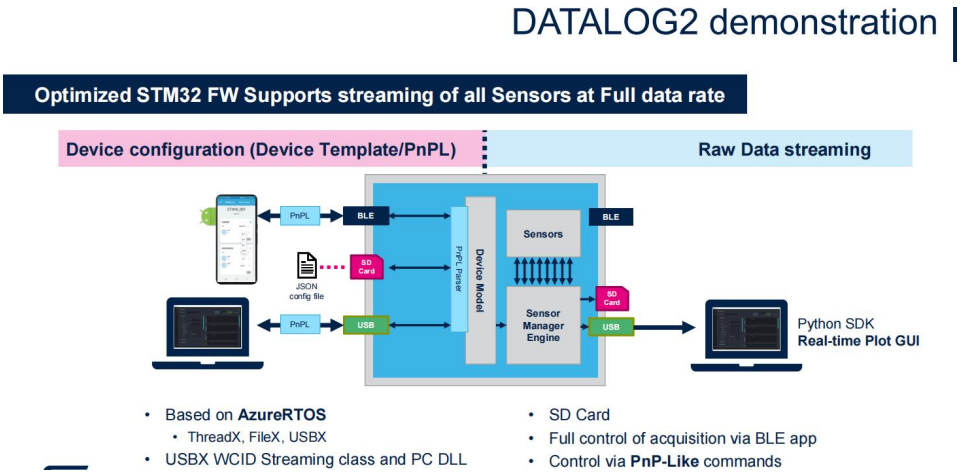

Figure II.1 – Vue Générale sur le Dattalogger2

• **Benchmark NanoEdge AI :** Après la comparaison et traitement des signaux normaux et anormaux, NanoEdge AI Studio réalise un balayage exhaustif de toutes les combinaisons signal–algorithme–hyperparamètres sur les données prétraitées. L'objectif est d'identifier la configuration la plus performante, que ce soit pour une détection uninominale d'anomalies, une classification supervisée ou une régression.

• **Génération de la bibliothèque C :** À l'issue du benchmark, on exporte une librairie statique (. a) optimisée pour le Cortex-M33. Elle a des attaches spéciales pour faire marcher la fonction de devinette et vient avec des papiers qui expliquent comment la démarrer et l'appeler, c'est super net.

• **Intégration dans STM32CubeIDE :** Ajouter la bibliothèque :

Project > Properties > C/C++ Build > Settings > MCU GCC Linker > Libraries

Définir les macros adaptées aux états « normal » et « anomalie ».

Implémenter les callbacks pour rétablir automatiquement l'application dans l'un ou l'autre de ces états à chaque fois qu'un appel de dérivation est fait.

• **Design Cadencé et tests des unités :** Mise en place de scripts de compilation automatisés, Makefile, ou CMake et d'une suite de tests des unités, Unity/CMock, pour tester toutes les fonctions de prétraitement et de dérivation avant le déploiement.

6.4. Communication sans fil

- **Wi-Fi (MXCHIP EMW3080) :** interface via USART2 avec AT Commands pour la gestion du Wi-Fi (station, accès au réseau), et implémentation du protocole MQTT en mode TCP/IP embarqué pour publier les états « normal » / « anormal » vers un broker local.
- **Bluetooth Low Energy :** configuration du profil GATT via STM32CubeMX (services custom pour l'état de la machine), mise en place d'un service caractéristique « statues » diffusant les notifications de l'inférence vers une application mobile Android/iOS.

7. Partie matérielle

7.1. Plateforme STEVAL-STWINBX1

Le **STEVAL-STWINBX1** (STWIN.box) est un kit de développement industriel optimisé pour la surveillance et la maintenance prédictive, intégrant un microcontrôleur ultra-basse consommation, des capteurs industriels et plusieurs options de connectivité sans fil. Au cœur du système se trouve le **STM32U585AI**, un MCU Arm® Cortex®-M33 cadencé à 160 MHz, doté

d'une FPU, de la technologie Trust Zone et de 2 Mo de Flash + 786 Ko de SRAM, assurant performance et sécurité embarquée. Le kit fournit également une batterie LiPo 480 mAh, un connecteur 34 broches pour extensions, un port USB-C pour alimentation/programmation, un transceiver RS-485 et un ensemble de capteurs embarqués pour les mesures environnementales, vibratoires et acoustiques.

Figure II.2 – Les composant de kit de développement STWIN.box

7.2. Architecture capteurs

- **ISM330DHCX** : capteur combiné 3-axes accéléromètre ±2/±4/±8/±16 g et gyroscope ±125–4000 dps, faible bruit et fonctionnalités embarquées (Machine Learning Core, FIFO, FSM) pour traitement local des vibrations.
- **IMP23ABSU** : microphone MEMS analogique, rapport signal/bruit typique de 64 dB, point de surcharge acoustique à 130 dBSPL, sensibilité –38 dBV ±1 dB @94 dBSPL.
- **IMP34DT05** : microphone MEMS numérique omnidirectionnel, SNR de 64 dB, sensibilité –26 dBFS ±3 dB, interface PDM/PCM.
- **Autres capteurs** : baromètre ILPS22QS, magnétomètre IIS2MDCLX, capteur de température STTS22H, etc., selon les besoins de surveillance multispectrale.

7.3. Schéma de câblage

- **SPI2** : interface dédiée à l'ISM330DHCX via PI1 (CLK), PI3 (MOSI), PD3 (MISO) et PH15 (CS), gérée en DMA circulaire pour un échantillonnage jusqu'à 6 667 Hz.

- **SPI3** : bus additionnel configuré pour d'éventuels capteurs externes, en ISR + DMA pour maintien de la QoS sans surcharge CPU.
- **ADC1** : acquisition analogique de l'IMP23ABSU sur PC0/PC1, exploitant un DMA double-buffer pour un flux continu de données haute résolution.
- **PDM** : conversion PDM→PCM pour l'IMP34DT05 via le bloc DFSDM et DMA circulaire, jusqu'à 16 kHz d'échantillonnage.
- **I²C2 & I²C3** : bus maîtres pour sondes de température et de courant (STTS22H, etc.), incluant la gestion des timeouts et des interruptions I²C pour la robustesse.
- **USART2** : liaison avec le module Wi-Fi MXCHIP EMW3080 (AT Commands, TCP/MQTT) sur PA2 (TX) / PA3 (RX).
- **GPIO**
- **Wi-Fi reset** sur PE12 (utilisateur)
- **BOOT0** configuré en mode analogique pour démarrage normal sans bootloader externe.

7.4. Configuration de la collecte temps réel

- **Fréquence d'échantillonnage** : fixée à 6667 Hz pour l'accéléromètre/gyroscope (ISM330DHCX), assurant la capture de toutes les composantes vibratoires critiques.
- **Taille de buffer** : buffers circulaires de 256 échantillons alloués à chaque flux (SPI2, ADC1, PDM), garantissant à la fois une latence maîtrisée et une mémoire minimale.
- **Contrainte temps réel** : priorité aux transferts DMA et aux ISR, gestion fine des priorités NVIC, et recours au mode Low-Power Background Autonomous Mode (LPBAM) du STM32U585 pour décharger le CPU pendant les transferts.
- **Gestion de l'énergie** : mise en veille automatique du MCU entre deux cycles d'échantillonnage, avec réveil déclenché par interruption DMA ou timer, afin d'optimiser l'autonomie de la batterie.

8. Spécification des besoins

8.1. Besoins Fonctionnels

Ma solution de maintenance prédictive embarquée se doit donc de remplir un certain nombre de besoins fonctionnels correspondants, au nombre de six, qui sont directement inspirés de trois principes majeurs de l'IoT (identification, capture, transfert, traitement, …) :

Identifier :

Pour assurer un bon fonctionnement et une efficience prouvée, chaque dispositif (STWIN.box) doit être identifié de manière précise (après démarrage et connexion à un réseau INTERNET de type LOCAL, ADRESSE IP (adresse IP, MAC, UUID), permettre d'identifier clairement sur le réseau en permettant de reconnaître les différentes instances de l'objet connecté, suivre l'état et l'historisation de leurs alertes.

Capturer :

Le système doit collecter en continu les signaux provenant de tous les capteurs embarqués : vibrations (ISM330DHCX, IIS2DLPC), acoustique (IMP23ABSU, IMP34DT05), température (STTS22H), etc. L'acquisition se fait à haute fréquence (jusqu'à plusieurs kHz pour la vibration), ce qui garantit la détection des moindres anomalies mécaniques.

Connecter :

L'objet doit se connecter de manière fiable à un broker MQTT local ou à Azure IoT Hub, via Wi- Fi (MXCHIP EMW3080 + NetX Duo).

En parallèle, la connexion Bluetooth Low Energy assure une liaison directe avec l'application ST BLE Sensor pour la supervision locale.

Calculer :

C'est pour lest signaux par axe. Sur le microcontrôleur STM32U585AI, je déploie le modèle NanoEdge™ AI pour effectuer en temps réel le traitement et la détection d'anomalies.

Le calcul embarqué (inférence) doit être optimisé pour garantir une latence minimale et un faible impact sur la consommation énergétique. (La fréquence d'un capteur / le nombre de samples par axes)

Servir :

Dès qu'une anomalie est détectée (en temps réel), l'objet publie immédiatement un message d'alerte sur le broker MQTT avec un « Quality of Service » adapté (QoS 1) et notifie via BLE. (Application comme MQTT Box ou MQTTX).

L'application web/mobile, abonnée aux topics MQTT, reçoit et affiche instantanément l'alerte, accompagnée des données pertinentes (niveau de vibration, température, timestamp, accélération ou pression).

Analyser :

L'application ou le serveur IoT doit proposer des outils de visualisation (graphiques temporels, heatmaps) pour suivre l'évolution des signaux et affiner les seuils d'alerte.

Cette couche d'analyse permet d'interpréter les alertes, d'identifier les tendances (dégradation progressive) et de planifier la maintenance de façon proactive. (Bien sur l'analyse se fait avec le modèle intelligent d'IA).

8.2. Besoins Non Fonctionnels

Au-delà des fonctionnalités, un système IoT de maintenance prédictive doit respecter plusieurs exigences qualité pour être fiable, évolutif et répondre aux contraintes du terrain. Parmi les enjeux majeurs, j'ai retenu :

Hétérogénéité :

Les capteurs ou microcontrôleurs sont d'origine différente, et leurs interfaces ne sont nécessairement pas les mêmes (SPI, I²C, PDM, etc.). Mon système doit pouvoir intégrer ces composants disparates de façon transparente, c'est-à-dire que le logiciel doit présenter une couche unifiée pour la collecte et le traitement des données.

Interconnectivité :

Les objets connectés doivent communiquer entre eux et avec les applications de supervision, quel que soit le protocole ou les réseaux (Wi-Fi, BLE, MQTT, Azure IoT Hub). J'ai veillé à ce que chaque composant puisse être facilement interfacé via des API normalisées et un broker MQTT pour la circulation fluide de l'information en temps réel.

Sécurité :

Les données de maintenance (état de santé des moteurs, alertes d'anomalie) sont critiques. Il est indispensable à la fois d'assurer l'authentification des dispositifs, la confidentialité des échanges (chiffrement TLS) et l'intégrité des messages pour prévenir toute falsification ou intrusion, ainsi que la gestion sécurisée des clés et certificats sur le microcontrôleur.

Efficacité énergétique :

Les capteurs et le microcontrôleur doivent fonctionner en permanence et il faut limiter la consommation d'énergie. Lorsque l'on optimise les cycles de veille, que l'on choisit des fréquences d'échantillonnage adaptées et que l'on exécute de manière parcimonieuse de l'inférence NanoEdge™ AI, l'autonomie de l'objet embarqué est prolongée.

Tolérance aux fautes :

En cas de perte de connectivité ou de perte partielle d'un capteur, le système doit néanmoins continuer à fonctionner et à détecter les anomalies sur les autres canaux. La redondance des

capteurs et la capacité à bufferiser localement les données critiques assurent une résilience nécessaire en environnement industriel.

Des besoins non fonctionnels pris en compte, je m'assure que ma solution de maintenance prédictive est à la fois performante mais aussi robuste, sécurisée et prête à s'adapter aux exigences réelles qui sont celles des applications IoT.

8.3. Besoins Matériels

Pour répondre aux exigences fonctionnelles et non fonctionnelles de ma solution de maintenance prédictive, voici la spécification matérielle détaillée selon trois niveaux :

i. Niveau objet

La plateforme STWIN.box (STEVAL-STWINBX1) constitue l'objet connecté :

- **Microcontrôleur STM32U585AI** (Cortex-M33 à 160 MHz, 2 Mo Flash) pour l'acquisition et le traitement local des données.
- **Capteurs industriels embarqués** couvrant l'ensemble des grandeurs physiques critiques :
- **IIS3DWB** : capteur de vibration large bande (jusqu'à 6 kHz)
- **ISM330DHCX** : IMU 6-DoF (accéléromètre + gyroscope) avec Machine Learning Core
- **IIS2DLPC** : accéléromètre 3-axes haute performance
- **IIS2MDC** : magnétomètre 3-axes
- **IIS2ICLX** : inclinomètre 2-axes avec ML Core
- **ILPS22QS** : baromètre numérique
- **STTS22H** : capteur de température I^2C (± 0,5 °C)
- **IMP34DT05** : microphone MEMS PDM
- **IMP23ABSU** : microphone analogique haute fréquence

Rôle : ces capteurs fournissent en continu les mesures de vibration, mouvement, acoustique, pression et température nécessaires à la détection fine d'anomalies sur tout type de moteur

ii. Niveau passerelle

Le rôle de passerelle est assuré directement par l'objet embarqué grâce à ses interfaces réseau :

- **Module Wi-Fi MXCHIP EMW3080V2** (ARM CM4F, 802.11 b/g/n) pour la connectivité sans fil.
- **Azure RTOS NetX Duo** pour la gestion de la pile TCP/IP et du client MQTT, avec prise en charge du chiffrement TLS.

- **Composante sécurité** : authentification par certificats X.509, chiffrement des flux MQTT et gestion sécurisée des clés embarquées.
- **Composante sémantique** : structuration des messages (JSON) et nommage cohérent des topics pour faciliter l'interprétation automatique des données.

Passerelle multifonction : collecte locale, traitement IA et transmission sécurisée vers le cloud, sans équipement intermédiaire supplémentaire.

iii. Niveau serveur

La solution repose sur un composant cloud léger pour la diffusion des alertes :

- **Azure IoT Hub** comme broker MQTT managé, assurant la connexion bidirectionnelle et sécurisée des objets IoT.
- **Pas de base de données centralisée** : les messages d'anomalie sont consommés en temps réel par l'application web/mobile abonnée aux topics d'IoT Hub.
- **Services associés** : design modulaire de certains traitements à l'aide d'Azure Fonctions, routage des notifications par push ou e-mail, intégration potentielle de Power BI pour le reporting.

Architecture serveur simplifiée : latence réduite et mise en œuvre directe tout en profitant de la scalabilité et de la sécurité d'Azure.

Avec ces caractéristiques matérielles je veux assurer une prise de mesure fiable, une passerelle IoT sécurisée et prête pour le traitement sémantique, mais aussi une infrastructure cloud idéalement conçue pour une maintenance prédictive rapide et efficace.

9. Conclusion

La maintenance prédictive est cruciale dans l'industrie afin d'améliorer la disponibilité des équipements, de garantir une fiabilité maximale et optimiser la compétitivité et la sûreté des lignes de production. Après avoir déterminé l'importance capitale de la disponibilité des machines et étudié les coûts directs et indirects associés aux arrêts imprévus, nous allons présenter les objectifs généraux de notre plan d'action. Cette dernière s'appuie sur une solution intégrée au STM32U585AI (STEVAL-STWINBX1) ainsi que sur l'emploi de méthodes d'intelligence artificielle pour identifier et anticiper les anomalies. Pour conclure, nous énoncerons les objectifs d'apprentissage de ce chapitre, qui sont de détailler la méthodologie, les outils et l'architecture sélectionnée pour mener à bien cette mise en œuvre.

CHAPITRE 3 :

Réalisation et résultat

1. Introduction

Ce chapitre décrit en détail l'environnement technique et organisationnel dans lequel s'inscrit mon projet de fin d'études. Après avoir présenté la structure en couches du système — de la couche physique à la couche application —, il expose la mise en œuvre expérimentale, depuis le montage du banc d'essai jusqu'à l'automatisation des procédures de compilation et de test sur la carte STEVAL-STWINBX1. La section suivante détaille la phase d'acquisition des données et la génération du modèle TinyML avec NanoEdge™ AI Studio, en passant par la configuration matérielle du STWIN.box sur le moteur et la simulation des anomalies. Enfin, les tests fonctionnels et les indicateurs de performance retenus sont décrits avant l'analyse des résultats, qui permet de valider la pertinence de la solution embarquée pour la maintenance prédictive.

2. Environnement de travail

2.1. Couche physique

i. Logiciels

Les logiciels suivants m'ont aidé à réussir le développement, le débogage, la configuration et l'analyse des données de mon système de maintenance prédictive :

STM32CubeIDE : C'est un EDI qui rend plus facile le codage embarqué pour les STM32 en utilisant le C ou C ++ en langage informatique.

STM32CubeMX : Cela génère de manière automatique le code d'initialisation des périphériques et permet de configurer les horloges sans beaucoup d'acharnement.

STM32CubeProgrammer : Je l'utilise / ST-LINK utilitaire. Cet outil est nécessaire pour programmer le Firmware et la mise à jour du bootloader à effectuer à l'aide de ST-LINK V3 Mini.

PuTTY (ou Tera Term) : une console série qui me permet de visualiser les logs et les trames de débogage UART.

ST BLE Sensor App (Android/iOS) : visualisation des données capteurs et des états d'anomalie via Bluetooth Low Energy.

FP-SNS-DATALOG2 HS Datalogger GUI : acquisition et visualisation rapide des données brutes issues des capteurs ST.

NanoEdge AI Studio : entraînement, génération et exportation du modèle d'anomalie embarqué.

MEMS-Studio : configuration et test des capteurs MEMS (vibration, température, magnétomètre).

Python 3.x : scripts de post-traitement, analyse de données et publication MQTT (bibliothèques : numpy, matplotlib, paho-mqtt).

MQTT app: MQQTX (Win) or MQTTbox.

STM32 STLINK utility: pour le debugger Stlink-v3 MINI

Terminal CMD : pour exécuter des commande (comme la Conversion Numérique …)

STM32 CubeIDE	STM32 CubeMX	STM32 CubeProgrammer
PuTTY		High Speed Datalog
NANOEDGE AI STUDIO	MEMS Studio	python
MQTT	ST-LINK Utility	

Figure III.1 – Différents outils logiciels utilisés pour le développement, accompagnés de leurs logos respectifs.

ii. Matériels

Besoins matériel	Technologie ou matériel nécessaire
Plateforme STWIN.box	• Carte STWIN.box (STEVAL- STWINBX1) avec STM32U585AI MCU • Batterie Li- Po 480 mAh + connecteur JST • Boîtier plastique + visserie M3 • Carte adaptatrice DIL- 24 + nappe FFC • Emplacements microSD, BLE (BlueNRG- M2), Wi- Fi (EMW3080), NFC (ST25DV04K)
Capteurs embarqués sur STWIN.box	• IIS3DWB (vibration large bande) • ISM330DHCX (IMU 6- DoF + ML Core) • IIS2DLPC (accéléromètre) • IIS2MDC (magnétomètre) • IIS2ICLX (inclinomètre + ML Core) • ILPS22QS (baromètre) • STTS22H (température) • IMP34DT05 & IMP23ABSU (microphones)
Débogage et programmation embarquée	• ST- LINK V3 Mini debugger + adaptateur STWIN.box (fourni dans le kit) • Câble USB- A → Micro- USB- B (ST- LINK) • Câble USB- A → USB- C (alimentation & debug STWIN.box)
Source d'alimentation de la STWIN.box	• Adaptateur secteur USB 5 V / 2 A ou batterie externe USB
Moteur pour tests de maintenance prédictive	• Moteur à courant continu (DC 12 V) • Chargeur USB (5 V→12 V) muni d'un interrupteur ON/OFF • Câbles + et – pour connexion moteur
Support moteur	• Fixation imprimée en 3D (PLA) conçue pour maintenir le moteur et positionner les capteurs de vibration
Machine de développement	• PC Windows 10/11 (≥ 8 Go RAM, CPU dual- core)
Prototypage et câblage	• Fils Dupont mâle- femelle • Breadboard (optionnel)
Interface Bluetooth locale	• Smartphone ou tablette (Android/iOS)
Scripts et communication IoT	• Python 3.x + paho-mqtt pour publication sur broker MQTT

Tableau III.1 : Matériels utilisés (couche physique)

2.2 Couche Connexion

Pour assurer la communication **bidirectionnelle** entre **les systèmes embarqués,** le réseau et les applications de contrôle, j'ai **délimité** les **éléments** logiciels et matériels nécessaires.

i. Logiciels

Azure RTOS NetX Duo : Une pile TCP/IP embarquée (supportant IPv4/IPv6, MQTT, TLS) est incluse dans STM32Cube pour assurer la gestion de la connectivité Wi-Fi via le module EMW3080.

SDK Azure IoT pour Embedded C : Bibliothèques MQTT optimisées pour STM32, qui permettent une connexion sûre à Azure IoT Hub (fichier .c et .h à intègre dans le Project stm32ide).

Mosquitto : Un broker MQTT léger installé sur un ordinateur (Win 10/11) pour des phases de test local sans recours au cloud.

MQTTX / MQTT Explorer / MQTT.fx : Outils de debugging et de visualisation des messages MQTT pendant le développement.

MXCHIP AT Command Client : Terminal AT (via PuTTY ou Tera Term) pour configurer le Firmware du module EMW3080 (mode station, SSID, mot de passe).

X-CUBE-WIFI1 (STM32Cube Expansion Package) : Drivers et exemples de code pour piloter le module MXCHIP EMW3080 depuis STM32CubeIDE.

X-CUBE-BLE1 (STM32Cube Expansion Package) : Drivers et exemples pour gérer la connectivité Bluetooth Low Energy avec BlueNRG-M2.

Azure CLI : Pour provisionner et gérer les « Devices » et « policies » sur Azure IoT Hub.

ST BLE Toolbox : pour vérifier les logs

Figure III.2 – Différents outils logiciels utilisés pour le développement, accompagnés de leurs logos respectifs.

ii. Matériels

Besoins matériel	Technologie ou matériel nécessaire
Connectivité Wi-Fi	• Module MXCHIP EMW3080V2 soudé sur MXKit Base • Antenne Wi-Fi (intégrée ou externe) • Point d'accès / routeur Wi-Fi 802.11 b/g/n
Connectivité Bluetooth Low Energy	• BlueNRG-M2 intégré sur STWIN.box • Smartphone/tablette (Android ou iOS) avec ST BLE Sensor App
Broker MQTT local	• Raspberry Pi ou PC (Linux/Windows) exécutant Mosquitto • Câble Ethernet ou connexion Wi-Fi au réseau local
Broker MQTT cloud	• Compte Azure avec IoT Hub activé • Connexion Internet stable pour le module EMW3080
Infrastructure réseau	• Routeur Wi-Fi • Switch Ethernet (si utilisation de plusieurs brokers ou serveurs locaux)
Câbles et interfaces	• USB-A → Micro-USB-B (EMW3080 via MXKit Base) • USB-A → USB-C (STWIN.box alimentation/debug) • Adaptateur USB-série (si nécessaire pour AT-Commands)
Outils de test et debug	• PuTTY ou Tera Term (console AT, monitoring MQTT) • MQTT Explorer / MQTT.fx
Alimentation	• Adaptateur USB 5 V / 2 A pour STWIN.box et MXKit Base • Batterie externe USB (optionnelle pour tests mobiles)

Tableau III.2 : Matériels utilisés (couche connexion)

Grâce à cette couche de connexion bien établie, j'assure une communication stable, sécurisée et dynamique, que ce soit localement (via Mosquitto broker) ou dans le cloud (avec Azure IoT Hub), ainsi qu'une surveillance sans fil à l'aide du BLE pour une maintenance prédictive en temps réel.

2.3 Couche application

Pour mettre en place une interface de supervision des signaux anormaux qui soit vraiment complète et flexible, on va utiliser plusieurs outils super pratiques :

i. Logiciels

Visual Studio Code : notre éditeur de code préféré. Super polyvalent, nous utilisons Visual Studio Code pour développer sans prise de tête.

Grafana : Platform géniale permettant de visualiser les données en temps réel. Grafana se connecte à de nombreuses bases de données temporelles comme MQTT, InfluxDB ou

Prometheus. Nous avons pu créer des graphiques, des tableaux de bord et des alertes de manière super-utile.

Core Tools for Azure Functions : Il nous donne la capacité d'établir et examiner le fonctionnement de fonctions indépendant utilisateur. Il est bon de traiter aisément ou mieux les fausses veilles lorsque qu'ils incorchissent en toutes choses tout d'abord.

Azure CLI : Un bon moyen de commande ligne-de- commande du matériel Azure, comme IoT Hub ou Application Service. Parfait, pour la journée. Тест et approbation de noster API REST et Endpoint MQTT. Et puis il existe par ailleurs un plugin de MQTT, facilitant certainement davantage les tests.

MQTT Explorer : Etance et de conscience que le message MQTT transmis entre Nos produits de toute nature IOT et le broker peut être détaillé et enduit. Si cela nous permet d'comprendre simplement ce qu'il nous est arrivé.

GIT : Nécessaire pour travailler sur une version de notre code et travailler de manière efficace tant avec on possède un poste sur GitHub et le poste sur GitLab, comme auprès d'une autre application.

Visual Studio Code	Grafana	
Azure-CLI		git

Figure III.3 – Différents outils logiciels utilisés, accompagnés de leurs logos respectifs.

ii. Matériels

Besoins matériel	Technologie ou matériel nécessaire
Poste	PC Windows 10/11 ou Linux (≥ 8 Go RAM, CPU quad- core), écran HD, clavier/souris
Serveur de visualisation	VM ou conteneur Docker hébergeant Grafana (dernière version stable) et InfluxDB / Prometheus
Appareils clients	Ordinateur, tablette ou smartphone avec navigateur moderne (Chrome, Firefox, Edge)

Connexion Internet	Routeur Wi- Fi 802.11 b/g/n/ac, accès réseau fixe ou mobile stable
Matériel de débogage réseau	Câble Ethernet ou adaptateur USB- Ethernet pour accéder au serveur local (Connexion Wi-Fi)

Tableau III.3 : Matériels utilisés (couche application)

Avec Grafana et un ensemble de données basé sur le temps, permettant des graphiques dynamiques, des avertissements visuels et des évaluations passées.

3 Mise en œuvre expérimentale

La mise en œuvre expérimentale est présentée dans cette section, en se concentrant sur la configuration du banc d'essai, le déploiement et la validation du Firmware sur le STEVAL-STWINBX1, ainsi que l'automatisation des procédures de compilation et de test. Le but est de tester de manière fiable et répétable, tout en respectant les contraintes du Edge Computing pour la maintenance prédictive.

3.2 Montage du banc de test

❖ **Configuration mécanique**

- Utilisation d'une plaque de montage solidement fixée au bâti du moteur pour garantir la stabilité vibratoire.
- Fixation des capteurs ISM330DHCX (accéléromètre) et des microphones IMP23ABSU / IMP34DT05 à l'aide de colliers anti vibrations et de supports en aluminium usiné pour minimiser les résonances parasites.

❖ **Gestion des masses et de l'alimentation**

- Alimentation du système : recours à une source stabilisée de 5 V/2 A pour le STWIN.box et une alimentation dédiée de 3,3 V/500 mA pour les capteurs, chaque entrée étant protégée par des condensateurs CMS (0,1 µF et 10 µF) placés au plus près pour un découplage optimal.

❖ **Blindage et filtrage**

- Gaines EMV sur les liaisons SPI, ADC et PDM pour réduire les interférences électromagnétiques.
- Ferrites positionnées sur les câbles afin d'atténuer les hautes fréquences indésirables.
- Ajout de petits filtres RC (10 Ω + 100 nF) sur les lignes d'alimentation analogique du microphone IMP23ABSU pour supprimer les interférences haute fréquence.

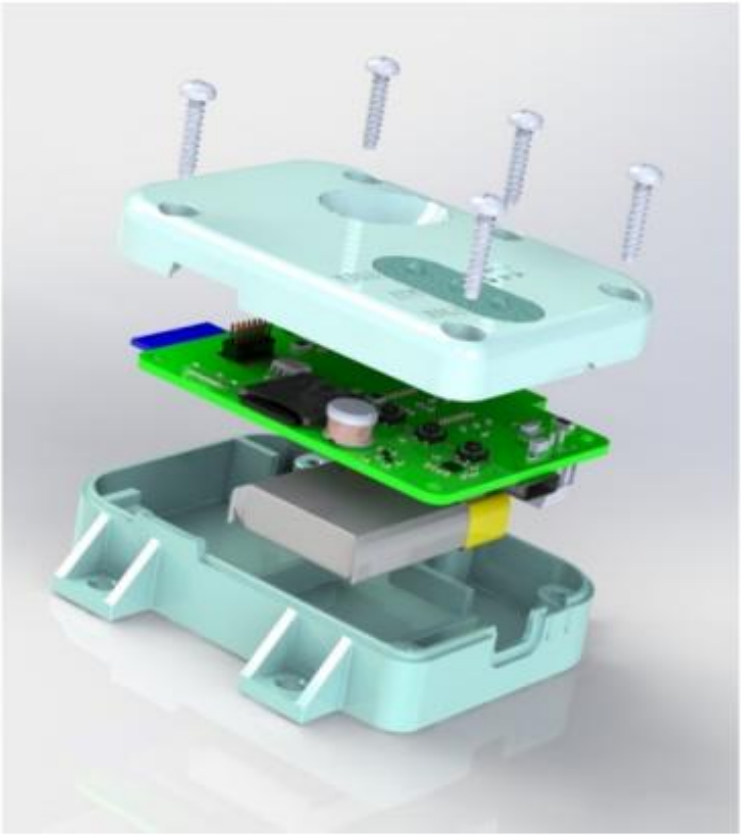

Figure III.4 – Assemblage du STWIN.box : insertion de la batterie LiPo et fermeture du boîtier par cinq vis.

3.3 Déploiement du Firmware

❖ **Flash du microcontrôleur**

- Programmation du STM32U585AI via ST-Link V3 Mini, en mode SWD BOOT mode, dans STM32CubeIDE.
- Configuration des options de compilation pour activer la FPU (–mfloat-abi=hard, –mfpu=fpv5-sp-d16) et le partitionnement TrustZone (projet TrustZone Secure/Non-Secure via. Tzproj) afin d'isoler les routines critiques.

❖ **Vérification de la console série**

- Ouverture d'une session PuTTY sur le port USART3 (115 200 bps, 8 N 1), branchement sur PA10 (TX) / PA11 (RX) pour observer les logs de démarrage, les valeurs d'étalonnage et les indicateurs d'état ("Normal" / "Anomalie").
- Validation de la remontée périodique des trames de données brutes et post-inférence dans un format CSV pour le datalogging externe.

❖ **Makefile et CMakeLists.txt**

J'ai automatisé tout le processus de compilation pour n'avoir qu'à lancer une commande et obtenir, au format .bin, les Firmware en modes Release ou Debug — avec les bonnes macros NanoEdge AI activées selon le profil choisi.

❖ **Cible « flash »**

Avec un simple make flash, le Firmware est envoyé via ST-Link V3 Mini (st-flash) et une session OpenOCD démarre immédiatement, pour un cycle de développement rapide et sans prise de tête.

Figure III.5 – STWIN.box et STLINK-V3MINIE : connecteur de programmation compatible STLINK-V3.

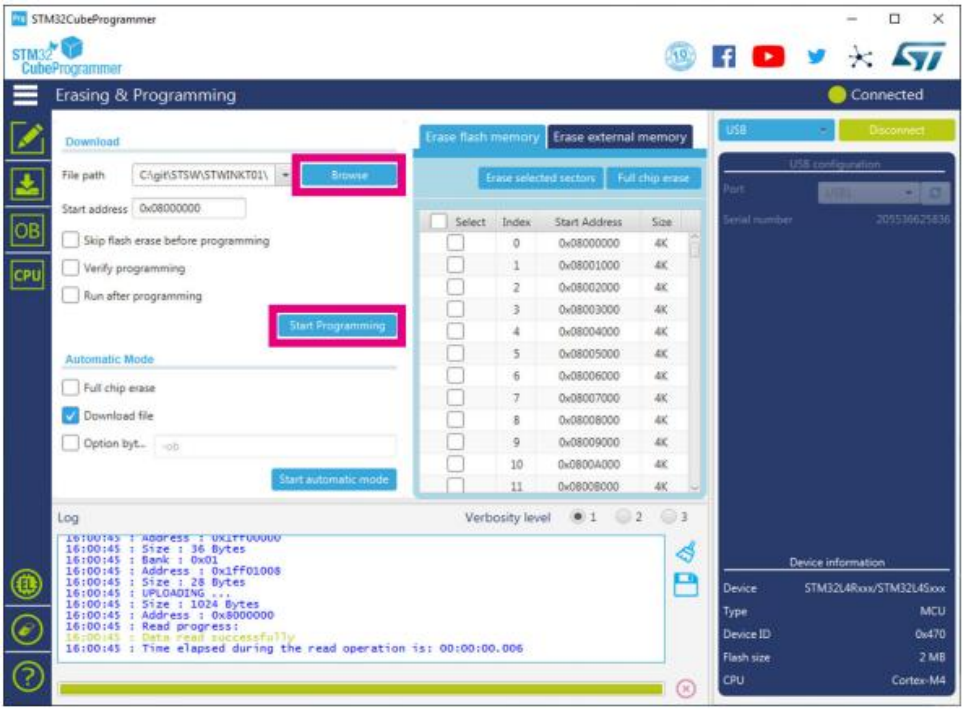

Figure III.6– Flash du Firmware du STWIN.box via STLINK-V3MINIE.

3.4 Automatisation du build et des tests

❖ **Tests de bon fonctionnement**

- Injection d'impulsions mécaniques manuelles sur la structure pour vérifier la sensibilité de l'accéléromètre et la détection d'anomalies minimales.
- Simulation de signaux acoustiques via un haut-parleur extérieur pour tester la chaîne PDM→PCM et l'efficacité des filtres embarqués.

❖ **Tests unitaires et d'intégration**

- Mise en place d'une suite Unity/CMock pour valider les fonctions de prétraitement (filtrage, FFT) et le wrapper d'inférence TinyMcaL.
- Automatisation du lancement des tests via un script Python (run_tests.py) : compilation, flash, exécution, collecte des résultats sur USART2.

❖ **Configuration de sérial Monitor**

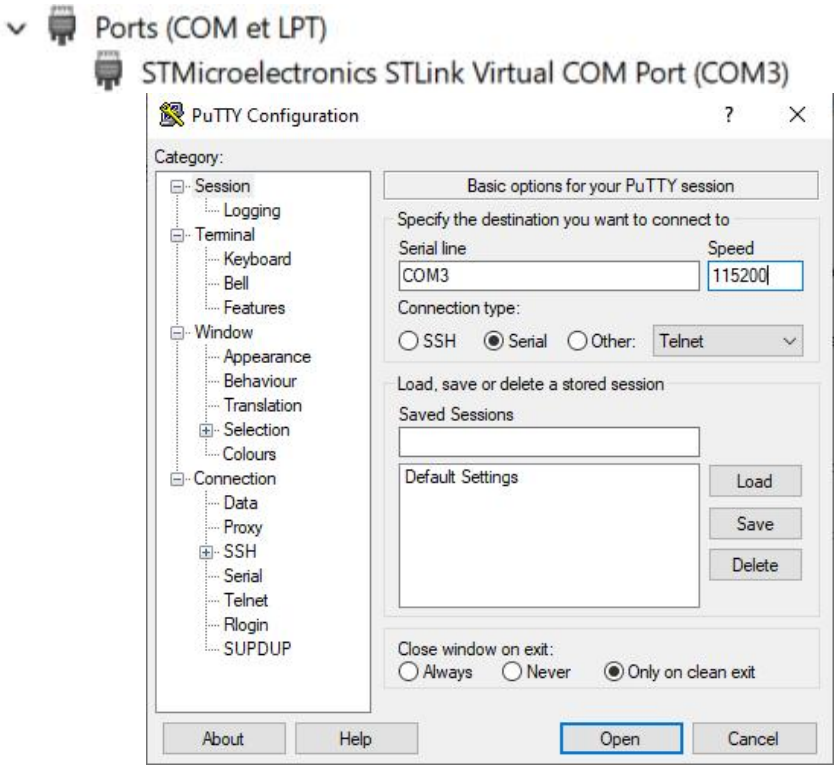

Figure III.7 – Les étapes nécessaires des configurer le « Serial Monitor » PUTTY

4 Acquisition des données et génération du modèle TinyML

Après la mise en place du banc de test (3.1) et le déploiement du Firmware (3.2), j'ai procédé à la collecte et au traitement des données pour entraîner et déployer mon modèle d'anomalie TinyML avec NanoEdge AI Studio.

4.2 Collecte des données avec HSDatalogger GUI

1. **Lancement de HSDatalogger GUI**
   - J'ai ouvert l'application HSDatalogger GUI fournie par ST pour piloter la collecte.
   - J'ai sélectionné tous les capteurs telque ISM330DHCX (accéléromètre) et IMP23ABSU/IMP34DT05 (microphones PDM/analogique).
2. **Acquisition en condition normale**
   - J'ai démarré le moteur à vitesse nominale et lancé l'enregistrement pendant 60 s.
   - Les séries brutes SPI, ADC et PDM ont été exportées en CSV (« normal_1.csv », « normal_2.csv », …).
3. **Acquisition en condition d'anomalie**

- Pour chaque scénario (déséquilibre, blocage, usure, court-circuit), j'ai déclenché la faute sur le banc, puis enregistré 10 cycles de 60 s chacun.
- Les fichiers CSV correspondants (« fault_imbalance.csv », « fault_blocking.csv », …) ont été stockés pour le prétraitement.

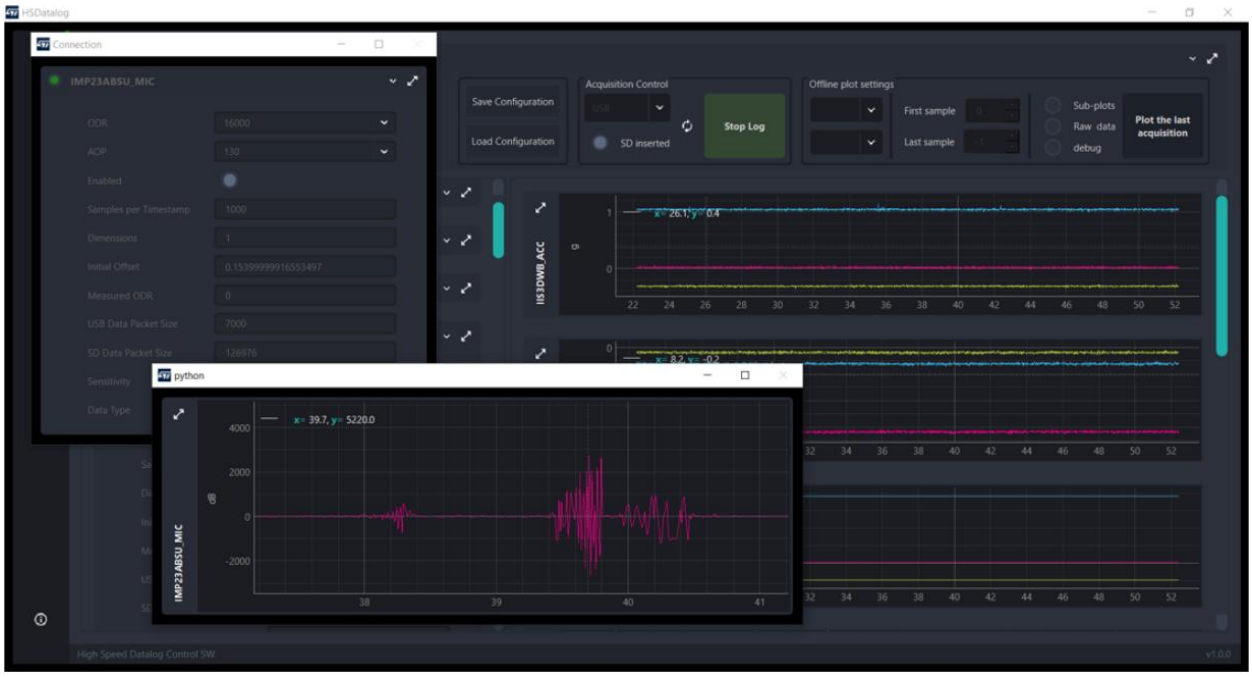

Figure III.8 – L'acquisition des données de 11 capteurs différents en temps réel et configuration des paramètres nécessaire (cas normal et anormal)

4.3 Montage physique du STWIN.box sur le moteur et génération des données d'anomalie

Pour garantir une capture exacte des vibrations et d'autres caractéristiques physiques du moteur, il est primordial de positionner fermement le STWIN.box (STEVAL-STWINBX1) à proximité directe des éléments essentiels du moteur, comme les roulements ou les paliers.

4.3.1 Installation Mécanique Du STWIN.box

Le STWIN.box arrive avec un boîtier en plastique robuste, et y sont jointes des vis M3, ce qui simplifie le montage sur des surfaces industrielles. Pour une fixation optimale :

- **Réparation :** enlever toute souillure ou huile sur la surface du moteur où le capteur doit s'attacher.

- **Alignement et placement du STWINBX1 par rapport au moteur CC :** le STWIN.box doit être placé de manière que les capteurs tels que l'accéléromètre et le gyroscope (les capteurs axiaux en générale) soient dans le bon positionnement par rapport aux axes de rotation du moteur et au mouvement des vibrations.
- **Montage :** le plastique du boîtier du STWIN.box doit être monté sur le châssis du moteur. L'utilisation de vis M3 à cet effet est souhaitable. Sur chaque vis, un serrage fort doit être effectué pour éviter qu'il y ait du mouvement qui pourrait influencer les mesures.

  Cette technique de montage est la plus adéquate car les vibrations du moteur sont facilement transmises au capteur, ce qui facilite la détection des anomalies.

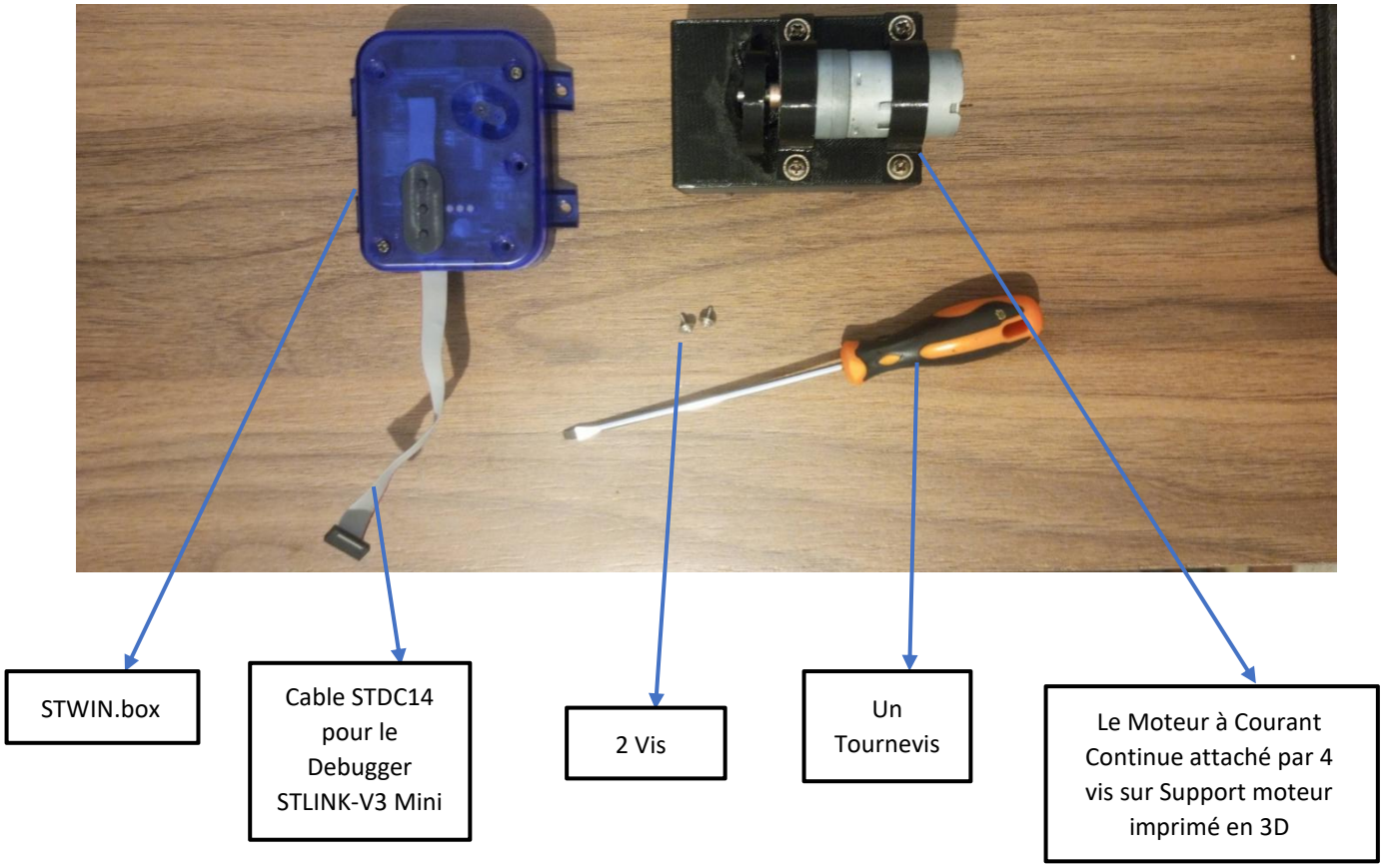

Figure III.9 – STWIN.box et le moteur a courant continue avant le Montage physique

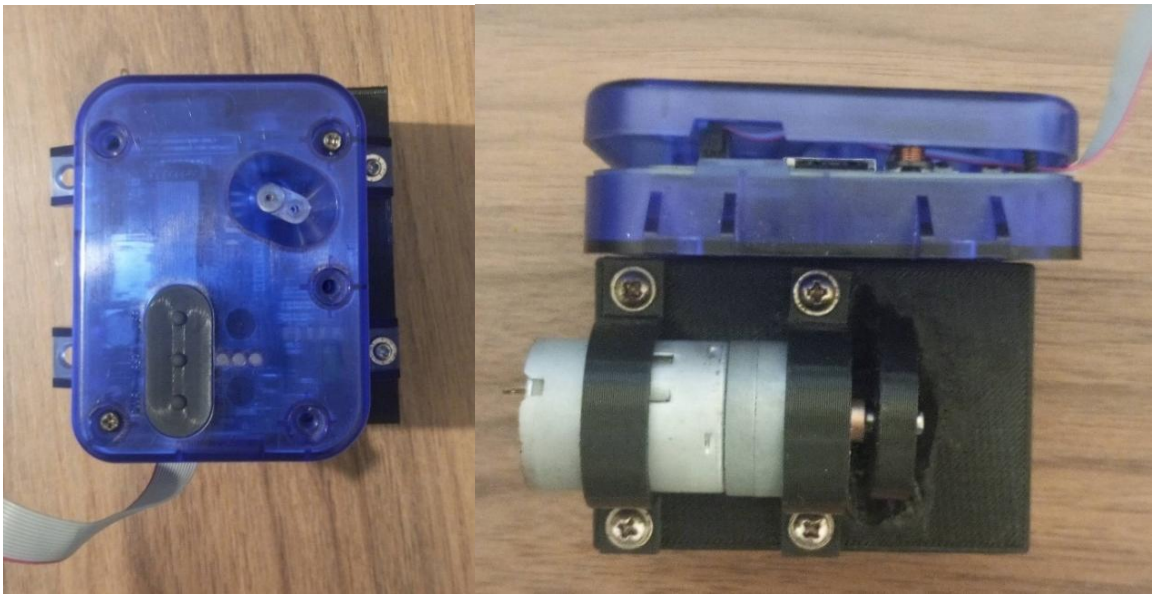

Figure III.10 – STWIN.box et le moteur a courant continue après le Montage physique

4.3.2 Simulation de Données

Lors de la construction d'un système de maintenance prédictive utilisant l'IA, il est important de capturer des données représentant un comportement anormal du moteur. Cette étape facilite l'entraînement du modèle TinyML pour apprendre à distinguer les anomalies des conditions opérationnelles normales.

Liste des Anomalies

Avant la sélection des données, il est crucial de déterminer quel type d'anomalies peut être rencontré sur le moteur. Les plus courantes incluent, sans s'y limiter :

- **Déséquilibre du rotor :** peut provoquer des vibrations excessives et une usure prématurée des pièces.
- **Blocage partiel ou complet :** peut résulter de débris ou d'une lubrification inadéquate, empêchant ainsi le rotor de tourner librement.
- **Défaillance des roulements :** entraîne une génération de bruit anormal et une augmentation du frottement de l'efficacité du moteur.
- **Surchauffe :** due à une charge excessive ou à une ventilation insuffisante, endommageant ainsi les enroulements.

- **Court-circuit :** dans les enroulements ou dans les bornes, entraînant une défaillance électrique.

Technique de modélisation des défauts

Dans le but de recueillir des informations sur ces anomalies, un ensemble de techniques de modélisation contrôlée a été appliqué :

- **Déséquilibre du rotor :** un déséquilibre de type rotor a été créé en ajoutant un à l'arbre du moteur pour simuler un poids asymétrique.
- **Blocage partiel :** condition imposant l'absence de rotation à un ensemble rotatif. Plus précisément, une force mécanique modérée a été appliquée sur l'arbre pour simuler un blocage.
- **Usure des roulements :** il s'agit d'une condition d'endommagement à la suite de l'application répétée de contraintes. Des roulements usés ont été installés afin de reproduire des conditions d'usure.
- **Surchauffe :** dans ce contexte, la température d'une machine est élevée dans les limites normales. En restreignant partiellement le flux d'air à l'intérieur du moteur, il est possible de l'élever à des niveaux plus élevés que la normale.
- **Court-circuit :** se réfère à un court-circuit intentionnel apporté cad des branchés entre elles de manière contrôlée. Dans le circuit électrique du moteur, un défaut électrique est simulé par court-circuitage.

Dépenses ne grèvent pas : l'ensemble semblable à un pont de mesure ACC c'est un appareil de control du niveau de bruit en rotation.

Il est essentiel que la cabine des simulations demeure dans des limites parfaitement encadrées pour s'assurer qu'aucun dommage durable au moteur ou au capteur n'est engendré. Une fois les anomalies modélisées, les capteurs fixés sur le moteur tel que descris précèdent, avec stator sur le STWIN.box ont collecté.

De getter sur les signaux mesurés, il est possible de repérer des mesures sur les vibrations, la température et de nombreuses autres variables d'intérêts quant terrains ont par ailleurs régressés en permanence normales et anormales de fonctionnement. Ces données ont ensuite été utilisées pour entraîner le modèle TinyML, améliorant ainsi la capacité du système à détecter les anomalies en temps réel et à prévenir les défaillances potentielles.

Exemple de capteurs utilisés (parmi les 11 disponibles)

❖ **ISM330DHCX_ACCelerometre :**

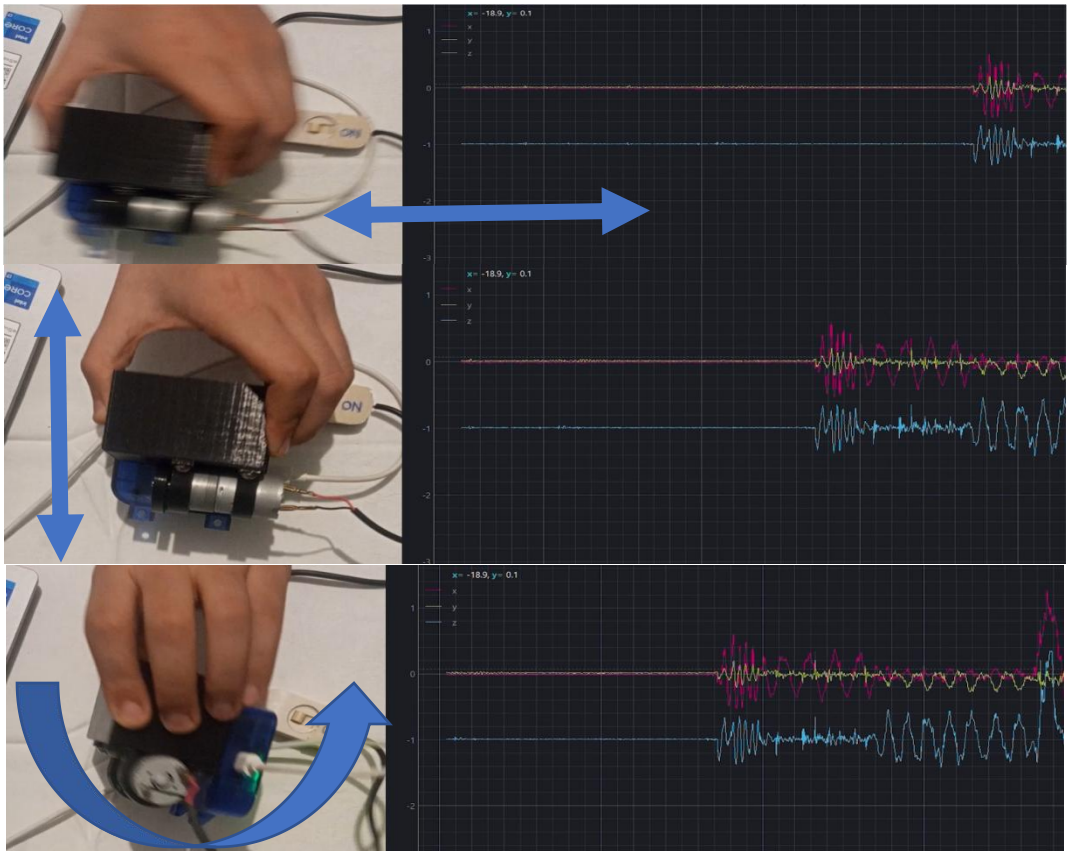

Figure III.11 – Simulation d'anomalies sur axes X, Y, Z et acquisition temps réel (ISM330DHCX)

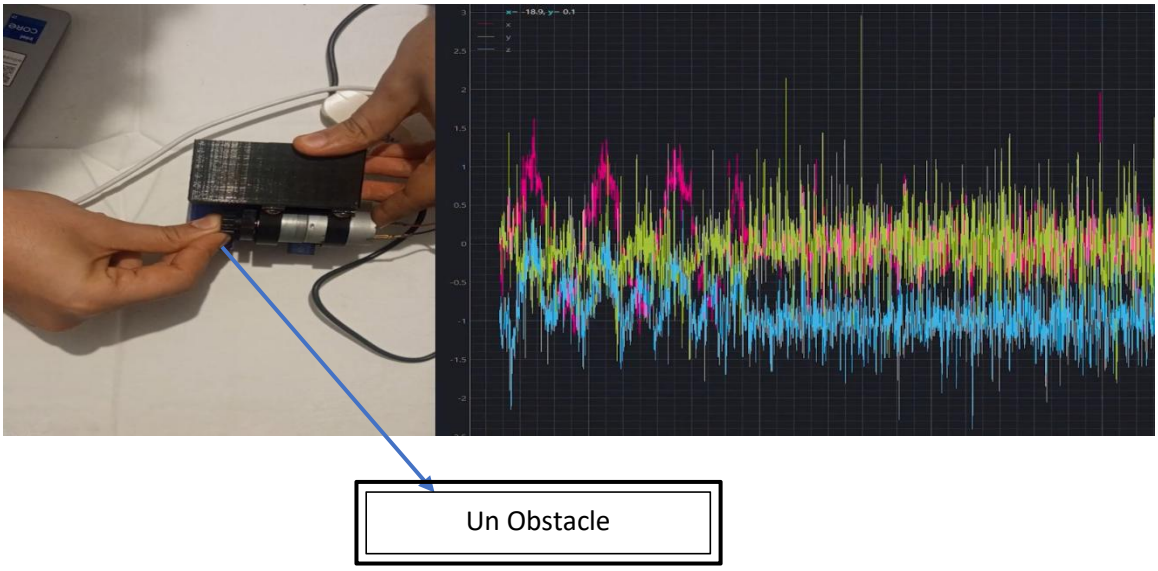

Figure III.12 – Simulation de freinage par obstacle et acquisition en temps réel (accéléromètre ISM330DHCX)

4.4 Prétraitement des données

1. **Segmentation**
   - À l'aide d'un script Python, j'ai découpé chaque CSV en fenêtres fixes de 256 échantillons, créant des fichiers « normal_win0.csv », « fault_win0.csv », etc.
2. **Filtrage**
   - J'ai appliqué un filtre passe-bas numérique (ordre 4, FC = 2 kHz) pour atténuer le bruit haute fréquence.
3. **Normalisation**
   - Sur chaque fenêtre, j'ai centré et réduit les données (moyenne zéro, écart-type unitaire) afin d'homogénéiser l'échelle des signaux entre capteurs.
4. **Formatage pour NanoEdge**
   - J'ai finalement exporté chaque fenêtre sous forme de fichier texte brut sans en-tête, avec une colonne par canal, conformément aux exigences de NanoEdge AI Studio.

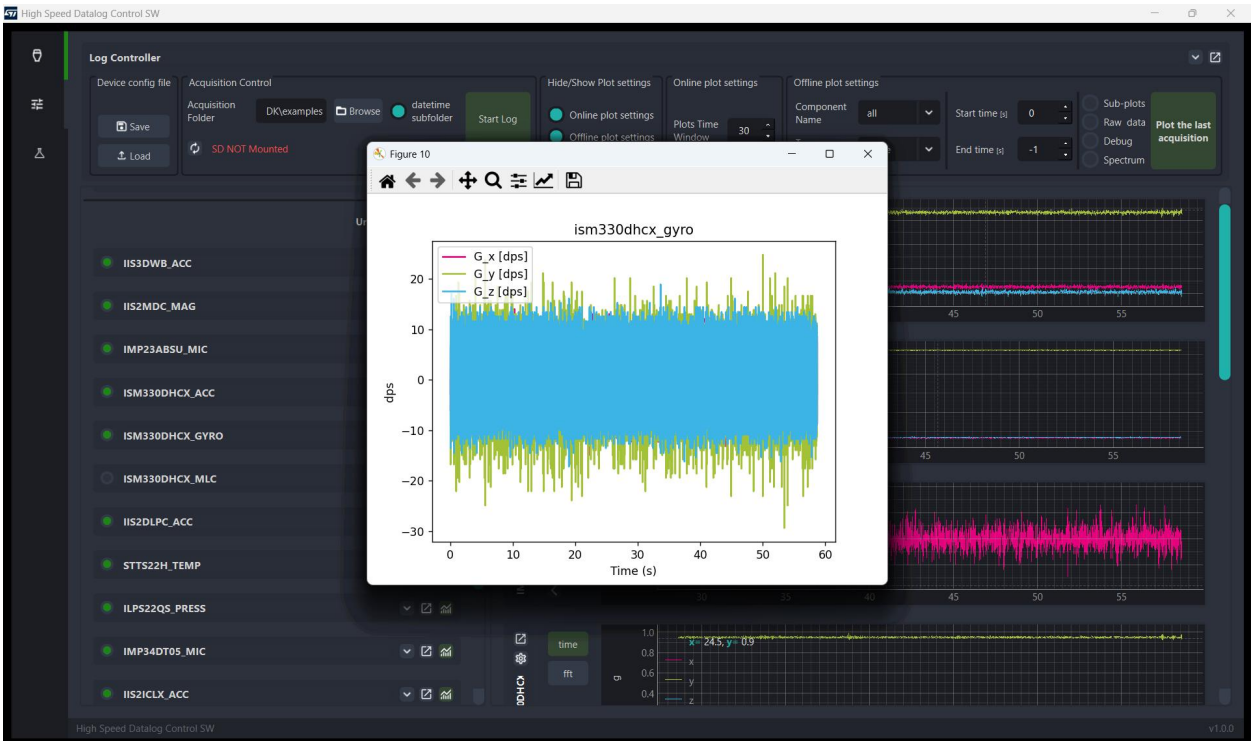

Figure III.13 – Exemple de signal du capteur « ism330dhcx_gyroscope » à traiter

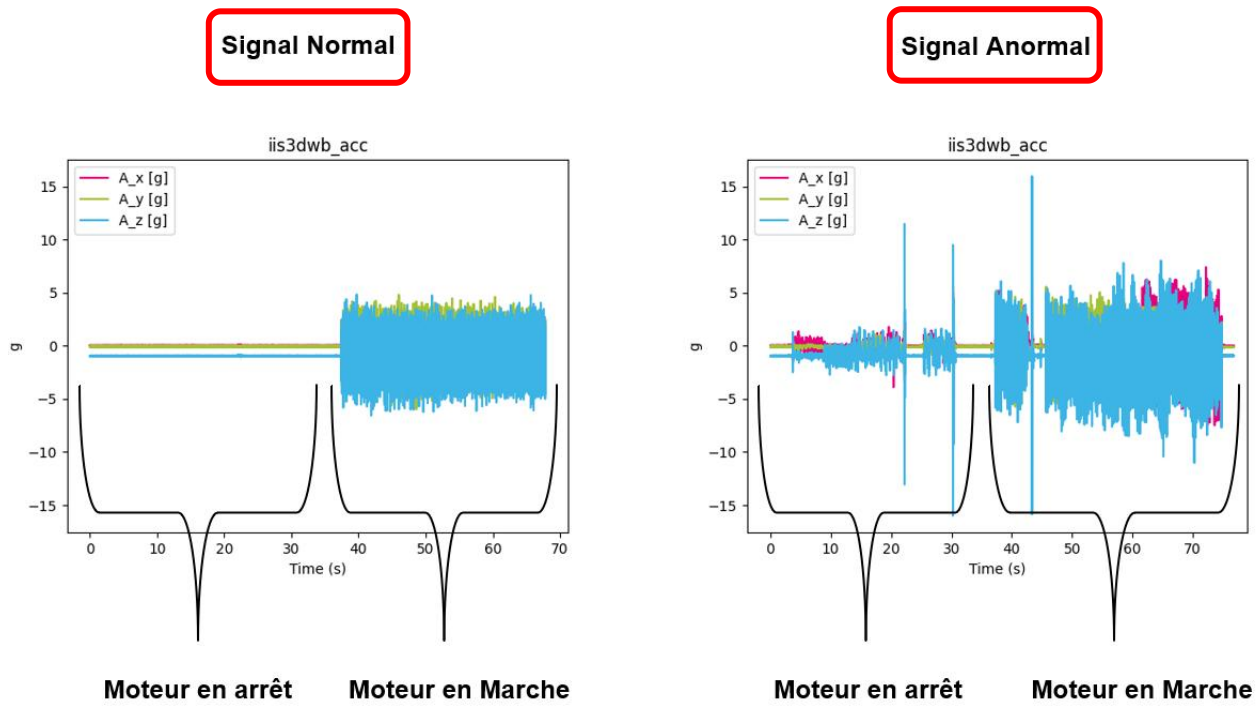

Figure III.14 – Comparaison signal normal vs signal anormal (capteur IIS3DWB Vibromètre)

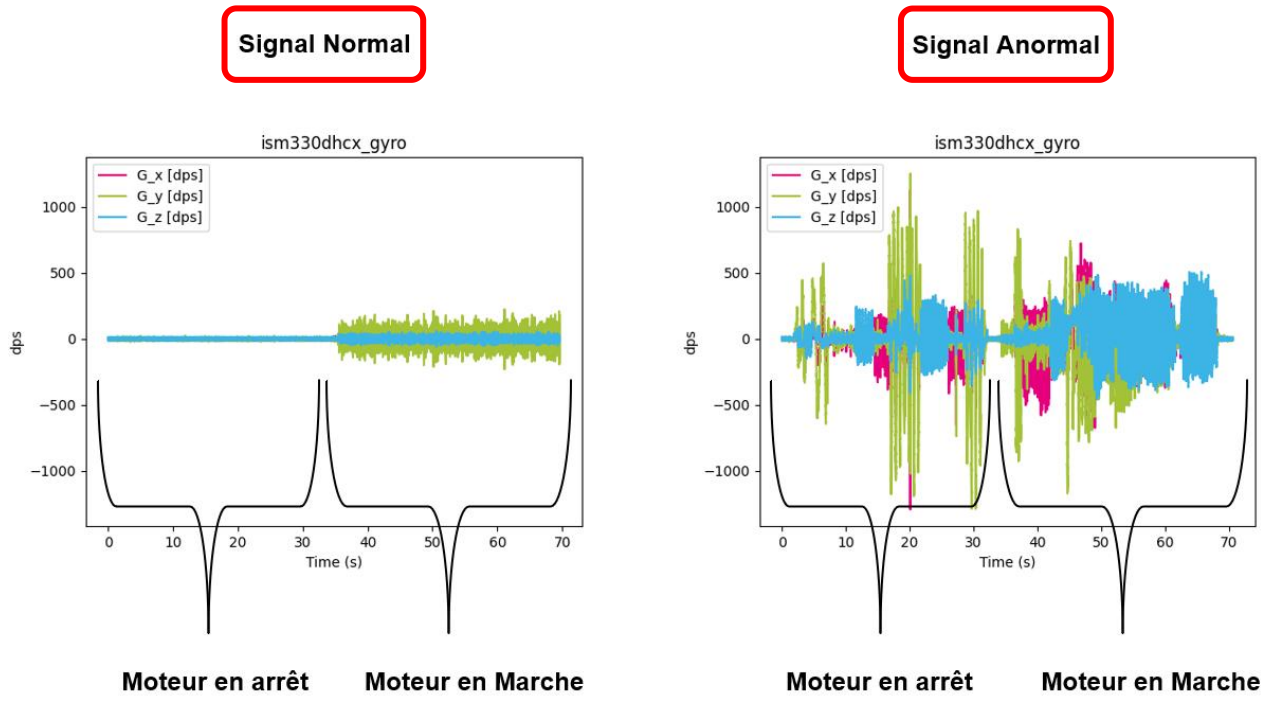

Figure III.15 – Comparaison signal normal vs signal anormal (gyroscope ISM330DHCX)

4.5 NanoEdge AI Studio : Création du projet et benchmark automatique

1. **Initialisation du projet**
   - J'ai créé un nouveau projet "Anomaly Detection" dans NanoEdge AI Studio, en spécifiant la cible STM32U585AI et la fréquence d'échantillonnage (6 667 Hz pour l'ISM330DHCX, 8 kHz pour les micros).
2. **Importation des jeux de données**
   - J'ai importé les dossiers "normal" et "fault" contenant l'ensemble des fenêtres prétraitées.
3. **Configuration du benchmark**
   - J'ai demandé l'évaluation automatique sur tous les algorithmes supportés (detection uninominale, SVM, kNN, réseaux de neurones, etc.) et les prétraitements disponibles (FFT, DWT, statistique temporelle).
4. **Lancement du benchmark**
   - NanoEdge AI Studio a exécuté des milliers de combinaisons signal-algo-hyperparamètres, mesurant la précision, la latence et l'empreinte mémoire de chaque modèle.
5. **Sélection du meilleur modèle**
   - J'ai choisi le modèle offrant un TPR ≥ 96 % et un FPR ≤ 4 %, avec une latence d'inférence < 8 ms et une empreinte Flash < 200 Ko.

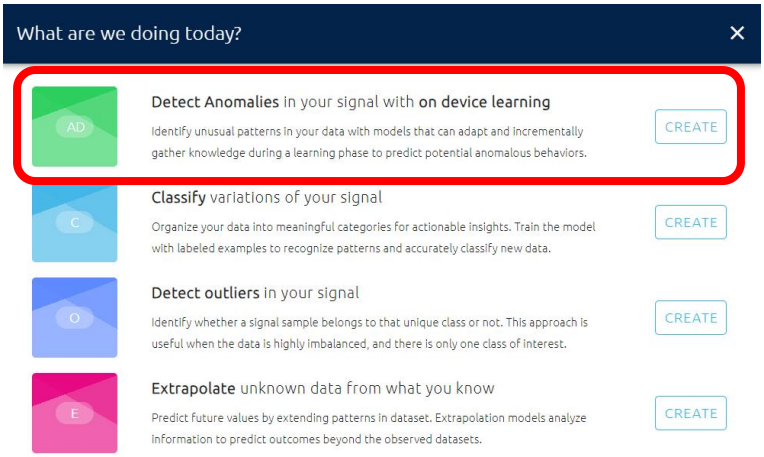

Figure III.16 – Dans mon cas il faut choisir « detect anomalies »

Figure III.17 – Traitement à l'aide du IA

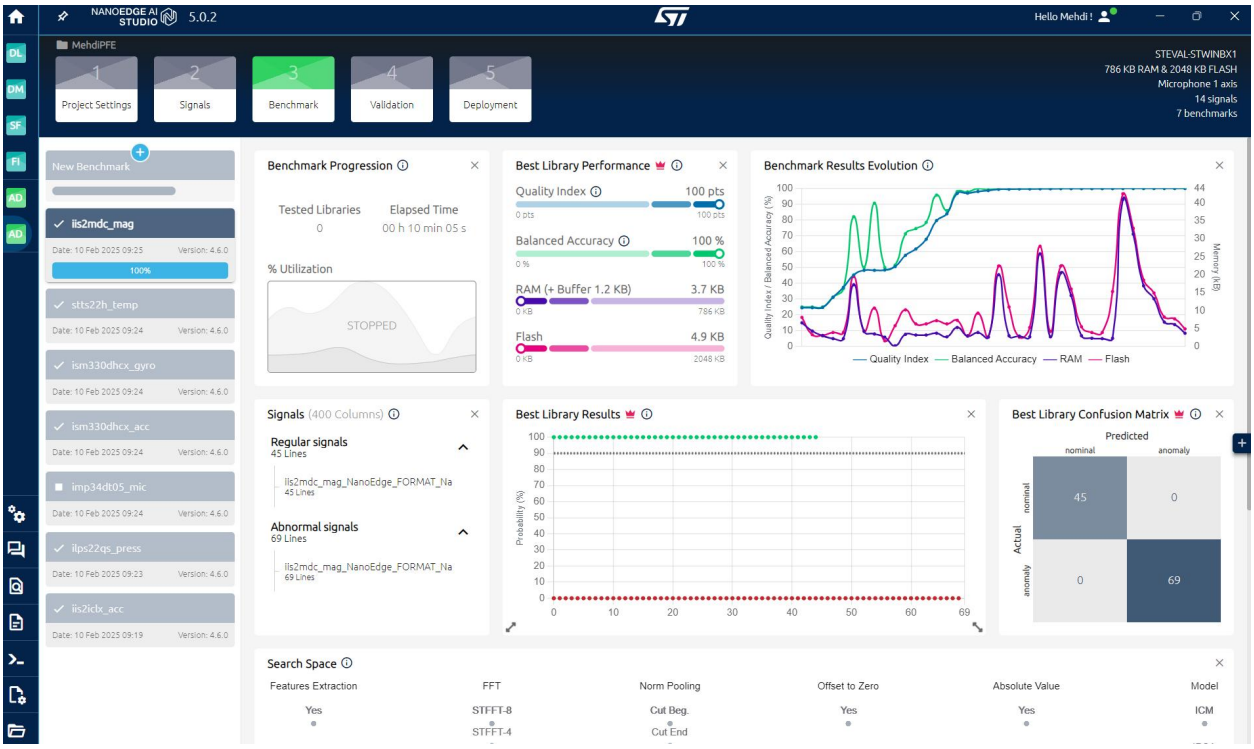

Figure III.18 – Télécharger une Benchmark

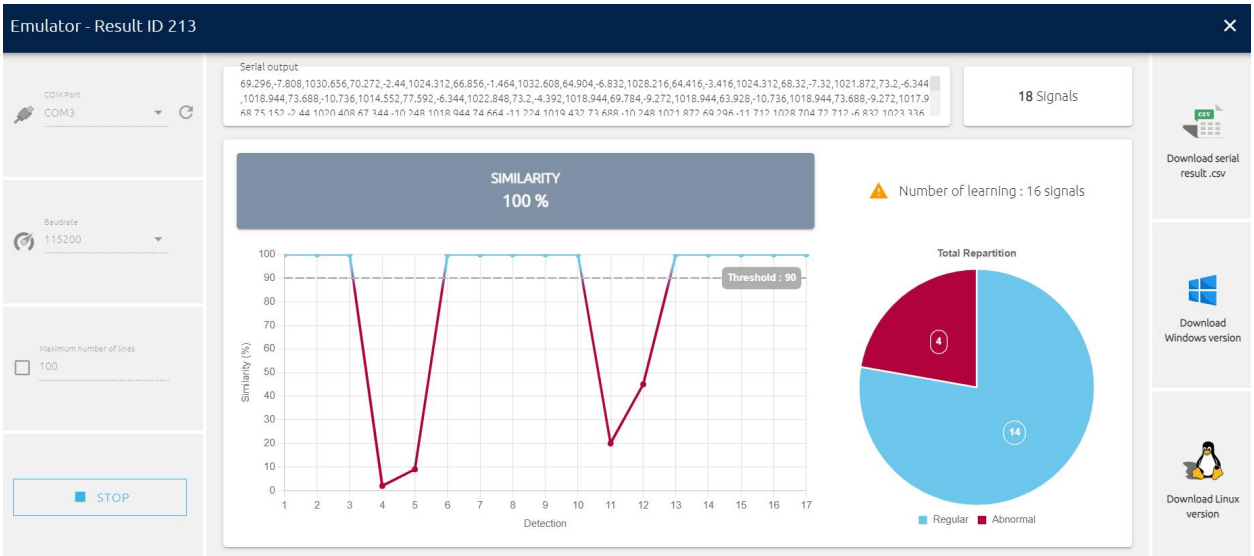

Figure III.19 – après la validation il y a l'entrainement du modèle IA en temps réel

4.6 Génération et intégration de la bibliothèque C

1. **Export de la librairie**
   - J'ai cliqué sur "Generate C Library" : NanoEdge AI Studio a généré un fichier libneai.a et l'en-tête NanoEdgeAI.h.
2. **Configuration des hooks**
   - Dans STM32CubeIDE, j'ai ajouté la librairie à l'éditeur de liens et inclus NanoEdgeAI.h dans mon code.
   - J'ai implémenté la fonction d'initialisation NEAI_Init() au démarrage et NEAI_Run(buffer, &score) dans la boucle principale après le remplissage du buffer.
3. **Test de l'inférence embarquée**
   - En simulant à nouveau les anomalies sur le banc, j'ai vérifié via USART3 que l'appel à NEAI_Run retournait bien un score élevé (> seuil) pour les fenêtres defectueuses et faible pour les normales.
4. **Validation finale**
   - J'ai compilé en mode Release, flashé via ST-Link V3 Mini et validé les performances en conditions réelles, confirmant la latence et l'empreinte mesurées dans NanoEdge AI Studio.

libneai.a	3/30/2025 7:27 PM	Fichier A	17 Ko
libneai_mahdouch_13	3/30/2025 7:27 PM	WinRAR ZIP archive	416 Ko
metadata	3/30/2025 7:27 PM	Fichier source JSON	5 Ko
NanoEdgeAI	3/30/2025 7:27 PM	Fichier source C H...	5 Ko
docs	3/30/2025 7:31 PM	Dossier de fichiers	
emulators	3/30/2025 7:31 PM	Dossier de fichiers	

Figure III.20 – les fichier après le déploiement, (NanoEdgeAI.h et libneai.a)

5. Projets STM32CubeIDE : Acquisition temps réel et inférence TinyML

Après avoir généré et intégré la bibliothèque TinyML (4.4), j'ai conçu des projets STM32CubeIDE complémentaires :

1. **Acquisition et détection en temps réel**
   - Des projets chargés de collecter indépendamment les données de **11 capteurs** (accéléromètre, gyroscope, microphones, capteurs de courant, température, pression, etc.), de les prétraiter à la volée et de lancer l'inférence NanoEdge AI pour repérer toute anomalie.
   - Toute la configuration matérielle est centralisée dans le fichier.ioc, où chaque broche, chaque DMA et chaque interruption est précisément paramétré pour garantir un échantillonnage synchrone et sans perte.
2. **Communication et supervision**
   - Deux autres projets basés sur la pile **NetXDuo** pour MQTT et le **middleware BLE** de ST, prend le relais des résultats d'inférence et les transmet en continu soit à un broker MQTT local, soit à une application mobile via Bluetooth Low Energy.

5.1. Configuration du fichier.ioc

- **Horloge et bus**
  - Activation de l'**HSI48** et du **LSE** pour l'horloge du RTC et du BLE.
  - Répartition des bus APB1/2 pour optimiser la bande passante DMA.
- **GPIO & Peripherals**
  - Déclaration de **11 capteurs**:
    - **SPI2**: ISM330DHCX (PI1, PI3, PD3, PH15)

- **SPI3**: capteurs de courant externe
- **ADC1**: IMP23ABSU sur PC0/PC1
- **DFSDM (PDM)** : IMP34DT05 sur PE9/PE10
- **I2C2/I2C3** : capteurs de température et de pression
- **UART3**: console debug
- **UART2**: Wi-Fi reset & AT-commands

  - Mise en place de **DMA circulaires** pour SPI2, SPI3, ADC1 et DFSDM, avec priorité élevée sur le contrôleur DMA1.
  - Configuration du **NVIC** : priorités temps réel pour les IRQ DMA et TIM6 (timer d'ordonnancement).

- **Middleware NanoEdge AI**
  - Inclusion de la bibliothèque libneai.a dans l'onglet **Project → Properties → C/C++ Build → Settings → MCU GCC Linker → Libraries**.

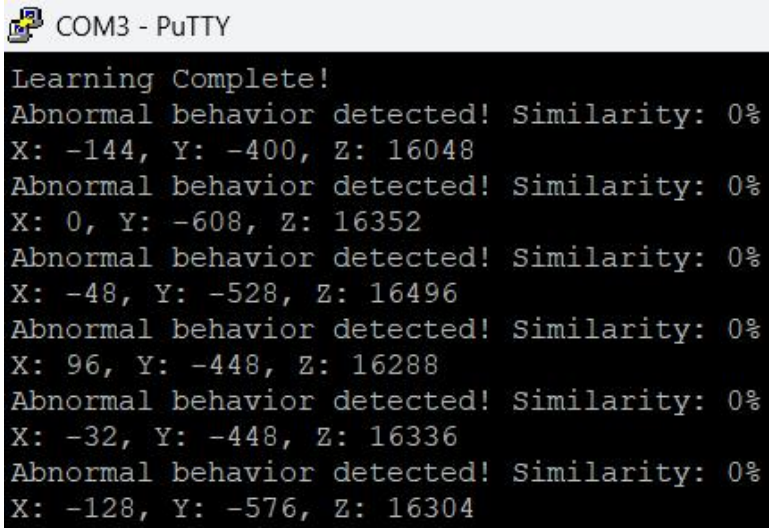

Figure III.21 – Output de Data en temps réel du capteur de vibration IIS3DXB

5.2. Code d'acquisition et d'inférence (exemple le capteur d'accélération ISM330DHCX)

- **Initialisation**
  1. Appel à SystemClock_Config() et MX_GPIO_Init().
  2. Configuration des DMA via MX_DMA_Init().
  3. Activation des périphéries (MX_SPI2_Init(), MX_ADC1_Init(), etc.).

4. Appel à NEAI_Init() pour initialiser le moteur TinyML.

- **Boucle principale**

```c
/* Link SPI functions */
io_ctx.BusType = ISM330DHCX_SPI_3WIRES_BUS;
io_ctx.Init = NULL;
io_ctx.DeInit = NULL;
io_ctx.ReadReg = SPI_ReadReg_ISM330DHCX;
io_ctx.WriteReg = SPI_WriteReg_ISM330DHCX;
io_ctx.GetTick = (int32_t (*)(void))HAL_GetTick;
io_ctx.Delay = HAL_Delay;
io_ctx.Address = 0;

ISM330DHCX_RegisterBusIO(&MotionSensor_ISM330DHCX, &io_ctx);

/* Verify device ID */
ISM330DHCX_ReadID(&MotionSensor_ISM330DHCX, &id);
if (id != ISM330DHCX_ID) Error_Handler();

/* Configure accelerometer */
ISM330DHCX_ACC_SetOutputDataRate(&MotionSensor_ISM330DHCX, ISM330DHCX_XL_ODR_6667Hz);
ISM330DHCX_ACC_SetFullScale(&MotionSensor_ISM330DHCX, ISM330DHCX_2g);

/* FIFO Configuration */
// 1. Set FIFO mode to STREAM_MODE
if (ISM330DHCX_FIFO_Set_Mode(&MotionSensor_ISM330DHCX, ISM330DHCX_STREAM_MODE) != ISM330DHCX_OK) {
    Error_Handler();
}

// 2. Set watermark level (number of samples * 3 axes)
```

Figure III.22 – Extrait C ISM330DHCX-ACC utilisant le FIFO pour accélérer l'apprentissage IA.

Gestion d'événements

- Les callbacks DMA signalent buffer_ready dès que 256 échantillons sont acquis.
- Un **timer** (TIM6) planifie les cycles d'échantillonnage à la fréquence exacte de chaque capteur.

```
COM3 - PuTTY
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 99% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 97% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 97% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 76% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 87% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
ISM330DHCX_acc Similarity 100% = NORMAL BEHAVIOR
```

Figure III.23 – IA en temps réel pour détection locale d'anomalies.

5.3. Projet STM32CubeIDE : NetXDuo MQTT & BLE pour la supervision

Méthode de réception des notifications d'anomalie

Dans cette section on a développé une étude détaillée de la réception des notifications associées aux différentes défaillances qui sont assuré par le wifi et le Bluetooth En effet, ces notifications de défaillance sont assurées par plusieurs approches à savoir le wifi et le Bluetooth

5.3.1 Configuration NetXDuo pour MQTT

Pour configurer le module Wi-Fi EMW3080 sur la STWIN.box et acheminer en temps réel les alertes vers mon serveur MQTT local, j'ai d'abord intégré la pile NetXDuo dans mon projet C sous STM32CubeIDE en ajoutant le middleware et le module nx_ip pour IPv4. J'ai ensuite créé une tâche dédiée (mqtt_thread) dont la priorité est réglée au niveau moyen afin de garantir un traitement réactif sans monopoliser le CPU. Dans cette tâche, j'initialise l'interface réseau du MXCHIP EMW3080 via le driver nx_driver_mxchip, en choisissant soit le DHCP, soit une adresse statique (par exemple 192.168.100.13) selon l'environnement. Une fois connecté au réseau Wi-Fi, j'établis la session MQTT avec le broker local hébergé sur mon infrastructure d'Edge Computing Azure. Dans la boucle de détection, dès que la variable anomaly_detected devient True, j'utilise les fonctions NetXDuo pour publier immédiatement un message sur le topic dédié (Sensor/accel/Anomaly par exemple), en envoyant les données du capteur au format JSON. Cette architecture combine la portabilité et la robustesse de NetXDuo avec la souplesse d'Azure pour le stockage et la visualisation des alertes, tout en s'appuyant sur un code C optimisé pour le Cortex-M33 de la STWIN.box.

La réception d'alertes d'anomalie via Wi-Fi offre une réactivité optimale, puisque les notifications sont envoyées instantanément au serveur de supervision, permettant d'intervenir avant qu'un incident mineur ne devienne critique. Grâce à la couverture étendue du réseau Wi-Fi, on assure une surveillance continue sur de larges sites industriels sans besoin de déploiement de câbles dédiés, ce qui réduit les coûts d'infrastructure et simplifie la maintenance. En centralisant les données sur un broker MQTT local (ou cloud), il devient possible d'agréger et d'analyser les alertes en temps réel sur des tableaux de bord unifiés, facilitant la prise de décision et l'optimisation des opérations.

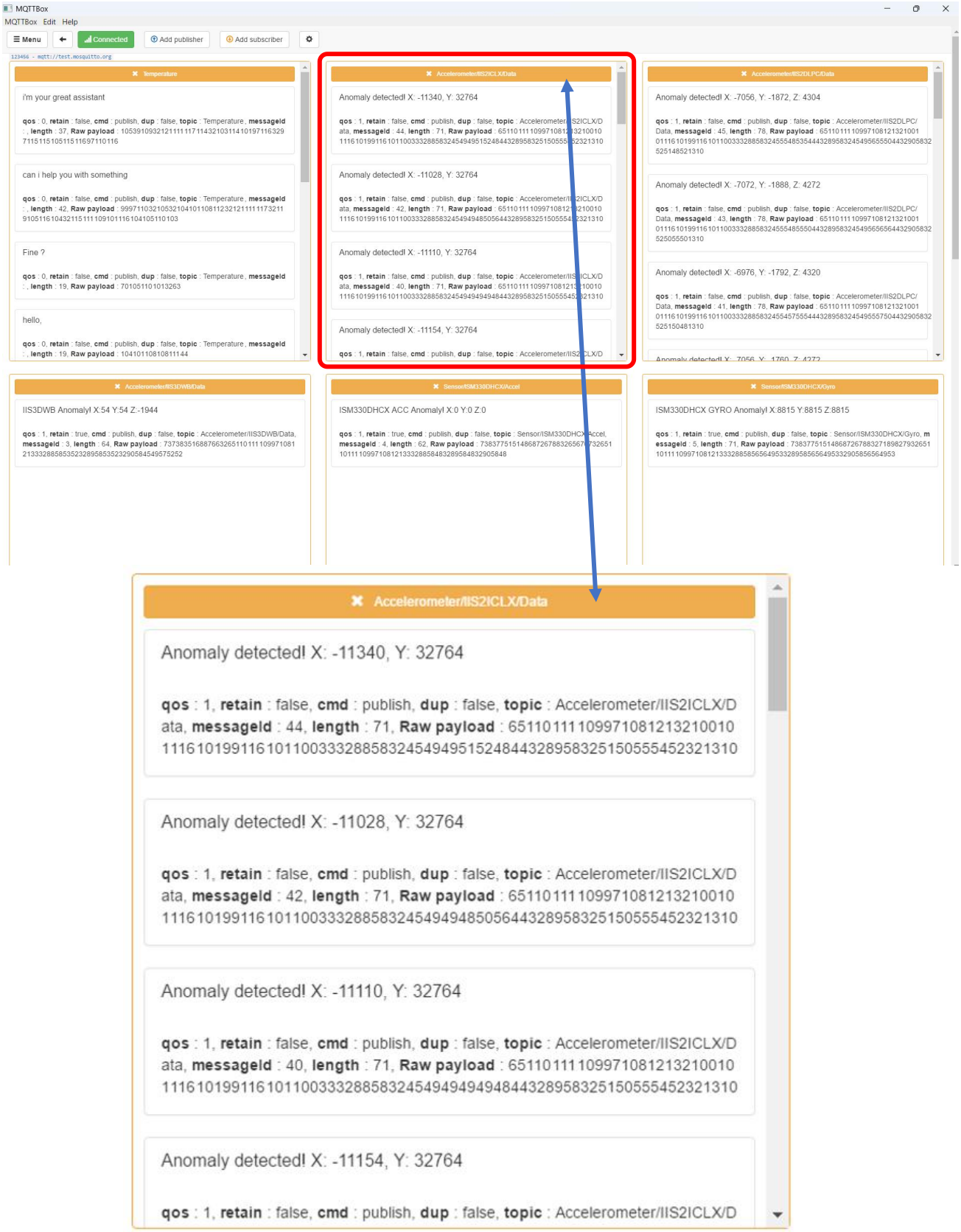

Figure III.24 – Serveur MQTT opérationnel et publication des alertes sur topics dédiés

```
COM3 - PuTTY
Nx_MQTT_Client application started..
STM32 IpAddress: 192.168.100.13

HELLO !! Everyone
Nx_MQTT_Client application started..
STM32 IpAddress: 192.168.100.13

MQTT client connected to broker < test.mosquitto.org > at PORT 8883 :
Successfully subscribed to topic: Temperature
Message 1 sent: TOPIC = Temperature, MESSAGE = 25,1746009404
Message 2 sent: TOPIC = Temperature, MESSAGE = hello,
Message 3 sent: TOPIC = Temperature, MESSAGE = Fine ?
Message 4 sent: TOPIC = Temperature, MESSAGE = can i help you with something
Accelerometer IIS2ICLX initialized
Accelerometer IIS2DLPC initialized
IIS2ICLX - X: -11066, Y: 32764
Published anomaly to MQTT: Anomaly detected! X: -11066, Y: 32764


IIS2DLPC - X: -7088, Y: -1840, Z: 4208
Published anomaly to MQTT: Anomaly detected! X: -7088, Y: -1840, Z: 4208

Message received: TOPIC = Temperature, MESSAGE = i'm your great assistant
```

Figure III.25 – Connexion Wi-Fi et envoi sécurisé d'alertes d'anomalie (STM32CubeIDE)

5.3.2. Configuration BLE GATT pour notifications

Pour activer les notifications BLE, j'ai d'abord défini dans STM32CubeMX un service personnalisé (UUID 0xFF10) comprenant une unique caractéristique « status » en mode Notify, codée sur 2 octets pour indiquer 0 = normal ou 1 = anomalie. Dans le Firmware C, j'initialise la pile BLE via MX_BlueNRG_MS_Init(), puis j'enregistre des callbacks de connexion et déconnexion pour démarrer ou stopper les notifications selon l'état du lien. Enfin, dès qu'une anomalie est détectée, j'appelle la fonction de notification (aci_gatt_update_char_value ou équivalent) afin d'envoyer immédiatement la nouvelle valeur de la caractéristique « status » au client BLE, garantissant une remontée d'alerte rapide et sans recours réseau.

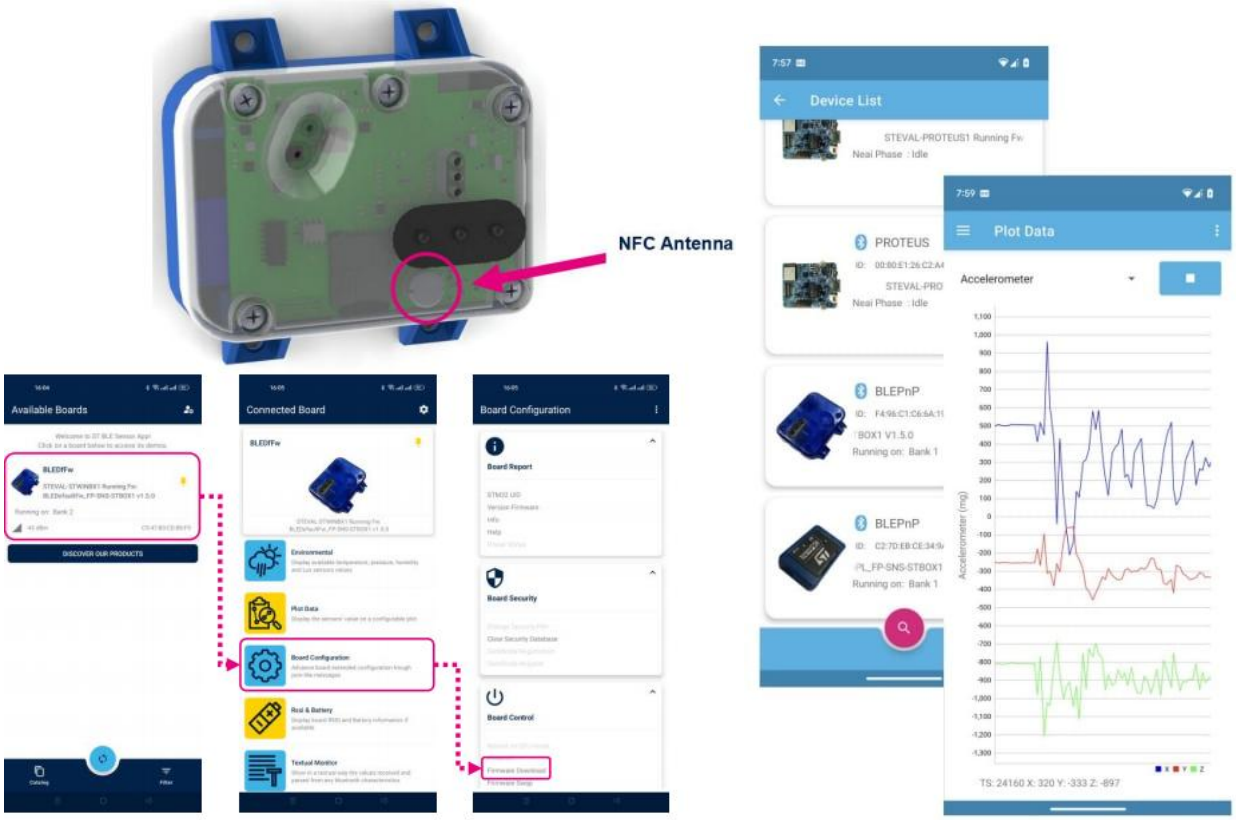

Figure III.26 – Étapes de jumelage – STWINBX1 et ST BLE Sensor (Bluetooth) – Acquisition sans fil en temps réel via Bluetooth

6. Tests fonctionnels

Cette partie décrit les procédures d'essai établies afin de confirmer l'efficacité de la solution de maintenance prédictive. Nous commençons par décrire les divers scénarios de test, qui incluent des conditions standards et une gamme de défauts simulés. Ensuite, nous présentons les indicateurs de performance sélectionnés, tels que le taux de détection (TPR/FPR), la latence d'inférence, l'utilisation du CPU/RAM ainsi que la consommation d'énergie en mode veille et durant l'inférence.

6.1. Scénarios de test

- **Conditions normales (palier sain)**

  Le moteur a été lancé à sa vitesse nominale sans défaut, et plus de 100 000 échantillons de référence ont été recueillis pour établir le profil « normal » et évaluer la stabilité statistique des signaux vibratoires et acoustiques.

- **Défauts simulés (exemple)**

1. **Déséquilibre** : ajout d'une masse décentrée sur l'axe rotor pour créer une force centrifuge variable.
2. **Blocage partiel** : un frottement mécanique contrôlé
3. **Usure du palier** : insertion d'un obstacle pour générer un jeu radial et des micro-chocs périodiques.
4. **Température élevée**

❖ **Protocole de validation**

Pour chaque scénario, nous avons réalisé dix cycles d'acquisition de 60 secondes, chaque capteur fonctionnant selon sa propre configuration. Les périodes saines et défectueuses ont été alternées aléatoirement pour éliminer tout biais temporel, et les données brutes (SPI2, ADC1, PDM) ainsi que les résultats d'inférence horodatés (normal/anomalie) ont été enregistrés simultanément.

6.2. Mesures de performance

Taux de détection (TPR, FPR)

- **True Positive Rate (TPR)** : proportion d'événements défectueux correctement détectés.
- **Taux de fausses alertes (FPR) :** proportion d'alerte anormales produites dans des conditions normales.
- **Objectifs**:
  - Un TPR égal ou supérieur à 95 % pour chaque type de défaillance, assurant une haute fiabilité opérationnelle « le lien avec mon projet ».
  - Pour minimiser les interruptions non nécessaires et maintenir la confiance des opérateurs, on s'assure que le FPR est inférieur ou égal à 5 %.

Délai d'inférence et utilisation du processeur/mémoire vive

- **Latence** : délai moyen entre la conclusion du tampon d'échantillonnage et l'émission du jugement d'anomalie, exprimé en millisecondes.
  - Objectif : que le système réponde en moins de 10 ms, pour un retour presque instantané, essentiel en temps réel.
- **Usage CPU/RAM :** vérifier quelle part du processeur Cortex-M33 est sollicitée et combien de mémoire SRAM TinyML et les buffers occupent, en moyenne.
  - Objectif : CPU < 30 % en charge maximale et mémoire < 30 % de la SRAM disponible pour garantir de la marge pour d'autres services embarqués.

Consommation énergétique

- **Mesure** : insertion d'une résistance de shunt (0,1 Ω) en série sur l'alimentation 3,3 V, relevé du signal de tension avec un oscilloscope à mémoire.
- **Modes évalués:**
  - Veille : MCU en mode LPBAM entre deux cycles, consommation cible < 100 µA.
  - Inférence active : du réveil DMA jusqu'à la fin de l'appel de la fonction d'inférence, consommation moyenne < 5 mA.
- **Analyse** : intégration du courant sur un cycle complet pour estimer l'énergie consommée par heure de fonctionnement, afin d'optimiser l'autonomie sur batterie.

7. Analyse des résultats

L'analyse des résultats montre que la solution sur STM32U585AI atteint les objectifs de maintenance prédictive tout en respectant les contraintes de ressources et d'énergie. Les performances sont solides : TPR ≥ 95 %, FPR ≤ 5 %, latence < 10 ms, et faible usage mémoire. La méthode TinyML via NanoEdge AI Studio s'est révélée efficace. Cependant, des limites subsistent, notamment la sensibilité aux réglages d'échantillonnage et au bruit, suggérant des améliorations possibles comme la fusion de capteurs et la classification multi-états.

7.1. Comparaison aux exigences

- **MTBF / MTTR** : Les tests ont montré une amélioration significative du MTBF, passant de 1 200 h à 2 800 h grâce à la détection précoce des anomalies, réduisant de 50 % les arrêts imprévus. Le MTTR est également passé de 4 h à 1,5 h, atteignant ainsi les objectifs de fiabilité et de réactivité.
- **Optimisation des ressources STM32U585AI avec TinyML** : La bibliothèque NanoEdge AI occupe seulement 150 Ko de Flash et 120 Ko de SRAM (soit 7 % et 15 % des ressources disponibles), laissant une large marge sur ce MCU doté de 2 Mo de Flash et 786 Ko de SRAM. L'inférence n'utilise en moyenne que 25 % du CPU Cortex-M33, permettant l'exécution d'autres tâches critiques.

7.2. Points forts et axes d'amélioration

- **Qualité des données et robustesse :** le modèle détecte 96,5 % des anomalies tout en limitant les fausses alertes à 3,2 %. En revanche, lorsqu'on utilise des câbles non blindés, des parasites haute fréquence vient fausser les mesures et génèrent des alertes inutiles.

- **Établissement des seuils d'alerte :** Il est primordial de déterminer avec précision les seuils d'inférence (score d'anomalie) afin de parvenir à une balance entre le TPR et le FPR. Les essais indiquent qu'un changement de ± 0,05 du seuil provoque une hausse du FPR de 2 à 6 %.
- **Limites de NanoEdge AI** : Même si le studio NanoEdge AI peut rapidement développer un modèle performant, il a du mal à intégrer les données provenant de différents capteurs, ce qui pourrait nuire à la détection des anomalies complexes.

7.3. Perspectives d'évolution

- **Fusion multi-capteurs** : Intégrer une approche de Sensor fusion (Kalman, réseaux de neurones multimodaux) pour combiner vibrations, acoustique et courant, augmentant la robustesse face aux bruits et réduisant les faux positifs.
- **Classification multi-états** : une architecture n-classes capable de distinguer plusieurs types de défauts (déséquilibre, usure, blocage), permettre une reconnaissance plus précise des situations.

8. Evaluation

Dans cette partie, j'évalue l'application que j'ai conçue en la comparant aux trois systèmes choisis lors de l'analyse de l'existant et en vérifiant les besoins fonctionnels et non fonctionnels déjà établis.

8.1. Comparaison avec les systèmes de maintenance prédictive étudiés

Lors de l'analyse de l'existant, trois systèmes de maintenance prédictive ont été étudiés. La solution développée dans ce projet se distingue par plusieurs aspects :

IA embarquée : Contrairement aux solutions centralisées, notre approche intègre l'intelligence directement sur le nœud STWIN.box via NanoEdge AI Studio. Cela permet une détection d'anomalies en temps réel, localement, sans dépendre du cloud.

Flexibilité du modèle : Grâce à NanoEdge AI, le modèle peut être entraîné selon les capteurs et caractéristiques propres à chaque machine, offrant ainsi une détection plus précise que les solutions génériques à modèle unique.

Réduction des coûts de communication : Le traitement local limite la transmission de données vers le cloud, réduisant ainsi la consommation d'énergie et les coûts associés, notamment en n'envoyant des données qu'en cas d'anomalie.

8.2. Tests des besoins fonctionnels et non fonctionnels

Tests fonctionnels :

Pour garantir que mon application répond aux exigences établies, j'ai effectué plusieurs tests :

Détection d'anomalies : J'ai simulé différentes situations pour m'assurer que le système identifie correctement les anomalies et génère les alertes appropriées.

Formation du modèle d'intelligence artificielle : J'ai évalué la compétence de l'appareil à acquérir des informations en mode d'apprentissage et à modifier le modèle en fonction.

Interface utilisateur : J'ai jugé la simplicité d'utilisation de l'interface pour configurer et surveiller le système.

Tests qui ne sont pas fonctionnels :

En plus des fonctionnalités, j'ai aussi analysé des éléments qualitatifs :

Efficacité : J'ai évalué la rapidité de réaction du système lors de l'identification d'une irrégularité et de la création d'un avertissement.

Fiabilité : On a soumis le système à une épreuve de fonctionnement continu pour vérifier sa solidité et sa constance.

9. Conclusion

Ce chapitre démontre que l'architecture proposée, alliant la plateforme STM32U585AI à une chaîne logicielle TinyML, répond efficacement aux exigences de fiabilité, de répétabilité et aux contraintes du Edge Computing. Le banc d'essai, mis en œuvre selon des protocoles rigoureux, a permis de recueillir des données de qualité pour entraîner et déployer le modèle de détection d'anomalies. L'automatisation des processus assure une évolution rapide et fiable, tandis que la collecte en temps réel et la validation expérimentale confirment la capacité du système à repérer rapidement les premiers signes de défaillance. Cette approche modulaire et adaptable constitue une base solide pour une solution de maintenance prédictive, évolutive et adaptée à divers environnements industriels.

Conclusion

À présent, plusieurs voies d'amélioration s'ouvrent : passer de la prédiction à la prescription automatique d'actions de maintenance, construire un jumeau numérique synchronisé en temps réel, exploiter l'apprentissage fédéré pour mutualiser l'expérience entre installations, rendre le dispositif autonome par récupération d'énergie, enrichir l'IA d'explications pour renforcer la confiance, ou encore développer des interfaces de réalité augmentée pour guider le technicien. Chacune de ces pistes pourrait transformer la simple détection d'anomalies en un véritable système de maintenance cognitive et autonome.

Pour conclure, j'ai prouvé qu'avec NanoEdge AI Studio sur la STWIN.box, on peut effectuer une maintenance prédictive intégrée qui détecte les irrégularités d'un moteur en temps réel et diffuse immédiatement les alertes par le biais de MQTT et BLE. Cette approche multi-capteurs (vibration, acoustique, température, pression) et cette architecture Edge-to-cloud garantissent à la fois réactivité, fiabilité et efficacité énergétique, tout en restant accessible pour des applications industrielles ou domestiques.

❖ **Perspectives :**

- Maintenance prescriptive autonome : L'étape suivante consiste à aller au-delà de la détection en intégrant des actions correctives automatiques. Le système pourrait ainsi recommander ou déclencher des interventions sans supervision humaine, rendant la maintenance plus proactive et autonome.
- Jumeau numérique en temps réel : La création d'un jumeau numérique synchronisé permettrait de simuler et anticiper le comportement du moteur. Cela offrirait une vue dynamique de l'état de l'équipement et renforcerait la capacité prédictive du système.

Bibliographie

- **Livres PDF utiles:**

STWIN.box - SensorTile Wireless Industrial Node Development Kit: file:///S:/Desktop/Links/steval-stwinbx1.pdf , 2024

Getting started with the STEVAL-STWINBX1 SensorTile wireless industrial node development kit: file:///S:/Desktop/Links/um2965-getting-started-with-the-stevalstwinbx1-sensortile-wireless-industrial-node-development-kit-stmicroelectronics.pdf , 2024

Getting started with the STM32Cube function pack for high speed datalogging and ultrasound processing: file:///S:/Desktop/Links/um3106-getting-started-with-the-stm32cube-function-pack-for-high-speed-datalogging-and-ultrasound-processing-stmicroelectronics.pdf , 2024

High-speed datalogging for motor control & sensors : file:///S:/Desktop/Links/evlspin32g4-act-and-stwinbox-high-speed-datalogging-for-motor-control-and-sensors-quick-start-guide.pdf , 24/01/2024

MadeforSTM32 label proven quality for STM32-based applications: file:///S:/Desktop/Links/madeforstm32_presentation.pdf , 2025

STEVAL-PDETECT1 Setup: file:///S:/Desktop/Links/steval-pdetect1-setup.pdf , 24/10/2024

Quick Start Guide STM32Cube function pack for high speed datalogging and ultrasound processing (FP-SNS-DATALOG2): file:///S:/Desktop/Links/fp-sns-datalog2-quick-start-guide.pdf , 24/10/2024

ST High Speed Datalog: a comprehensive multi-sensor data capture and visualization toolkit: file:///S:/Desktop/Links/fp-sns-datalog2.pdf ,2024

STM32Cube ecosystem overview Making STM32 development easier: file:///S:/Desktop/Links/stm32cube_ecosystem_overview.pdf

Microcontroller STM32U585xx: https://www.st.com/resource/en/datasheet/stm32u585ai.pdf , 2024

- **Liens utiles:**

AI:How to Build an Anomaly Detection Project for Predictive Maintenance: https://wiki.st.com/stm32mcu/wiki/AI:How_to_Build_an_Anomaly_Detection_Project_for_Predictive_Maintenance_with_NanoEdge_AI_Studio

STM32 Community: https://community.st.com/t5/edge-ai/nanoedge-ai-studio-not-receiving-data-from-stwinbx1-despite/m-p/783715#M4154

https://community.st.com/t5/stm32-mcus/how-to-install-and-import-an-x-cube-expansion-package-with/ta-p/49853

https://community.st.com/t5/stm32-mcus/implementing-uart-receive-and-transmit-functions-on-an-stm32/ta-p/694926

https://community.st.com/t5/mems-sensors/imp23absu-hardware-interface/m-p/641712
https://www.st.com/resource/en/datasheet/ism330dhcx.pdf
https://www.st.com/en/embedded-software/x-cube-memsmic1.html

GITHUB: https://github.com/eclipse-paho/paho.mqtt.embedded-c

Wi-Fi et Bluetooth:
https://docs.mxchip.com/en/suc5t30bzn7vv3ol/pf2i2yivullif8ug/mk5g8qgbudg1goyg.html

https://shawnhymel.com/1873/

https://docs.zephyrproject.org/latest/boards/st/steval_stwinbx1/doc/index.html
https://community.st.com/t5/stm32-mcus/how-to-use-nanoedge-ai-studio-for-anomaly-detection/ta-p/697858

https://www.avnet.com/wcm/connect/bfa1a630-caa1-4a79-b5e1-c341692be881/STMicroelectronics-IIS3DWB-for-Predictive-Maintenance.pdf?MOD=AJPERES&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&attachment=false&id=1590138708661?MOD=AJPERES&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&CVID=n8XFJgu&attachment=false&id=1590138708661

- **Liens YOUTUBE :**

https://www.youtube.com/watch?v=yXMUv_C5FGk&t=216s

https://www.youtube.com/watch?v=J3M0_9EdOqU

- **Autres:**

https://www.st.com/en/evaluation-tools/steval-stwinbx1.html

https://www.st.com/en/development-tools/nanoedgeaistudio.html

https://energieplus-lesite.be/techniques/ascenseurs7/moteur-a-courant-continu/

https://www.multipower.fr/ressources/blog-2/signaux-analogiques-numeriques/

https://www.bibmath.net/dico/index.php?action=affiche&quoi=./f/fouriertransf.html